CodeQuotient

Department of Computer Science & Engineering

LAB MANUAL

24CSE0212-Data Structures using Object Oriented Programming-II-2024-BE-CSE-4th Sem

Student Name	**RUPALI MANGLA**
Roll Number	
Course Name	**24CSE0212-Data Structures using Object Oriented Programming-II-2024-BE-CSE-4th Sem**

Faculty Incharge

_____

Table of Contents

S.No.	Aim	Pages
1	Factorial using recursion	8
2	Sum of all the digits using recursion	9
3	Prime factors using recursion	10
4	power(base, exp)	11
5	Form a new number	12
6	Binary equivalent using recursion	13
7	Greatest common divisor using recursion	14
8	Sort an array using merge sort	15 - 16
9	Sort an array using quick sort	17
10	Count the set bits of a number	18
11	Toggle the number except kth bit	19
12	Find the odd man out	20 - 21
13	Find the two strange elements	22
14	Saving the Earth with Binary Fever	23
15	Print message using class	24
16	Class with get and set functions	25
17	Count of all digits of a number	26
18	Calculate amount using compound interest	27

19	Convert given number of seconds to days, hours, minutes and second	28
20	Class - Rectangle	29
21	Class - TimeSpan	30 - 31
22	Class - Date	32 - 33
23	Sum of Range	34
24	Verify Prime Number	35
25	Java : toString() method - 1	36
26	Java : class - BankFee	37 - 38
27	Class - Circle	39
28	Class - TollBooth	40 - 41
29	Functions : Addition of numbers	42
30	Functions : Display of Strings	43
31	class - Box	44
32	Password too short Exception	45 - 46
33	Valid email address	47
34	Valid index of array	48
35	Java: Inheritance - MinMaxAccount	49 - 50
36	Java : Inheritance - Marketer	51
37	Inheritance : Actors	52 - 53
38	Java : Inheritance - 3	54

39	Inheritance : CalculateBill	55 - 56
40	Inheritance : BookCD	57 - 58
41	Inheritance : FilteredAccount	59
42	Inheritance : MemoryCalculator	60 - 61
43	Java : Interfaces - 1	62
44	Class - Writer and Book	63 - 64
45	Class - Writer and Book - 2	65 - 66
46	Class - Customer and Bill	67 - 69
47	Class - Customer and BankAccount	70 - 71
48	Class - MovablePoint and MovableCircle	72 - 74
49	Class : Circle and ResizableCircle	75 - 76
50	Duplicate the Queue elements	77
51	Get the mirror of the queue	78
52	Check for balanced parentheses in string	79 - 80
53	Flip the odd elements of queue	81
54	Number is a Happy number or not	82
55	Remove duplicates from a vector	83
56	Map contains same Range or not	84
57	Implement Own ArrayList in Java	85 - 86
58	Implement the stack using array	87 - 88
59	Reverse a string using stack	89

60	Implement the stack using Linked List	90 - 91
61	Design stack for push, pop and min functions	92 - 93
62	Find the next greater element on right side	94
63	Find the minimum bracket reversals for balanced expression	95 - 96
64	Evaluate the postfix expression using Stack	97 - 98
65	Evaluate the prefix expression using Stack	99 - 100
66	Implement the queue using array	101 - 102
67	Reverse a given queue	103
68	Reverse first N elements of a given queue	104
69	Print the List	105
70	Copy first list to second list	106
71	Move the Smallest and largest to head and tail of list	107 - 108
72	Check List for Palindrome	109 - 110
73	Find the loop in Linked list	111 - 112
74	Reverse a linked list	113 - 114
75	Add Two Numbers Represented by Lists	115 - 116
76	Delete a Node in Linked list given access to only that Node	117
77	Swap Two Nodes of Doubly Linked List	118 - 119
78	Rotate the Doubly Linked List by K elements	120 - 121
79	Rearrange the Even-Odd Nodes of Doubly Linked List	122 - 123

80	Given list is circular or not	124
81	Insert Nodes in a Circular Linked List	125 - 126
82	Delete in Circular Linked List	127 - 128
83	Count the Number of Nodes in Circular Linked List	129
84	Insert in a sorted circular linked list	130 - 131
85	Split the Circular Linked List in two parts	132 - 133
86	Create a binary tree from array	134 - 135
87	Print binary tree with level order traversal	136 - 137
88	Print nodes at odd levels of the binary tree	138 - 139
89	Write iterative version of inorder traversal	140
90	Complete the inorder(), preorder() and postorder() functions for traversal with recursion	141 - 142
91	Construct tree from given inorder and post order traversal	143 - 144
92	Count the number of leaf and non-leaf nodes in a binary tree	145 - 146
93	Print all paths to leaves and their details of a binary tree	147 - 148
94	Find the right node of a given node	149 - 150
95	Convert a binary tree into its mirror tree	151 - 152
96	Print cousins of a given node in Binary Tree	153 - 154
97	Print nodes in a top view of Binary Tree	155 - 156
98	Find out if the tree can be folded or not	157 - 158

99	Given two trees are identical or not	159 - 160
100	Find maximum depth or height of a binary tree	161 - 162
101	Evaluation of expression tree	163 - 164
102	Given binary tree is binary search tree or not	165 - 166
103	Find the kth smallest element in the binary search tree	167 - 168
104	Convert Level Order Traversal to BST	169
105	Find a lowest common ancestor of a given two nodes in a binary search tree	170 - 171
106	Find the floor and ceil of a key in binary search tree	172 - 173
107	Find K largest elements in array	174
108	Convert min Heap to max Heap	175 - 176
109	Check if a given tree is max-heap or not	177
110	Connect the sticks of different lengths with minimum cost	178 - 179
111	Implement Priority Queue using Linked List	180 - 181
112	Sort an array using heap sort	182 - 183

Aim: Factorial using recursion

Write a recursive function **factorial** that accepts an integer *n* as a parameter and returns the factorial of *n*, or *n!*.
A factorial of an integer is defined as the product of all integers from 1 through that integer inclusive. For example, the call of **factorial(4)** should return 1 * 2 * 3 * 4, or 24. The factorial of 0 and 1 are defined to be 1.
You may assume that the value passed is non-negative and that its factorial can fit in the range of type int.

Input Format:

The first line of input contains number of testcases , T.
Then T lines follow, which contains an integer,n.

Output Format:

For each testcase print the factorial in new line

Sample Input

2
4
3

Sample Output

24
6

Solution:

```
/*
 * Complete the function 'factorial' given below
 * @params
 *  n -> an integer whose factorial is to be calculated
 * @return
 *  The factorial of integer n
 */
int factorial(int n) {
  // Write your code here
  if(n==0 || n==1) return 1;
    return n*factorial(n-1);
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Sum of all the digits using recursion

Write a recursive function **sumOfDigits** that accepts an integer as a parameter and returns the sum of its digits. For example, calling sumOfDigits(1729) should return 1 + 7 + 2 + 9, which is 19. If the number is negative, return the negation of the value. For example, calling **sumOfDigits**(-1729) should return -19.
Constraints: Do not declare any global variables. Do not use any loops; you must use recursion. Also do not solve this problem using a string. You can declare as many primitive variables like ints as you like. You are allowed to define other "helper" functions if you like; they are subject to these same constraints.

Solution:

```
int sumOfDigits(int n)
{
  if(n==0) return 0;
    return (n%10)+ sumOfDigits(n/10);
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Prime factors using recursion

Write a program that accepts an integer *n* (where n >=2) and print all the prime factors of n using recursion.

Sample Input

24

Sample Output

2
2
2
3

Constraints : Do not declare any global variables. Do not use any loops; you must use recursion. You can declare as many primitive variables like ints as you like. You are allowed to define other "helper" functions if you like; they are subject to these same constraints.

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name
class Main{
   public static void prime(int n, int i){
      if(n<2){
         return;
      }
   if(n%i==0){
      System.out.println(i);
      prime(n/i,i);
   }
   else{
      prime(n,i+1);
   }
   }
   public static void main(String[] args)
   {
      // Write your code here
      Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
prime(n, 2);
   }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: power(base, exp)

Write a recursive function **power** that accepts two integers representing a *base* and an *exponent*, and returns the base raised to that exponent. For example, the call to power(3, 4) should return 3^4 i.e. 81. If the exponent passed is negative, then return -1.
Do not use loops or auxiliary data structures; Solve the problem recursively. Also do not use the provided library pow function in your solution.
Expected Time Complexity: O(log(n)); *here n denotes the exponent*
Input Format:
The first line of input conatins an integer T, denoting the number of test cases.
The second line of input contains 2 integers base and exponent seperated by space.
Output Format:
Print the answer when base is raised to the exponent.
Constraints:
-10 <= base <= 10
þÿ-15 <= exponent <= 15
Sample Input
2 // Test Cases
2 3
5 2
Sample Output
8
25

Solution:

```
class Result {
    static long power(int base, int exp) {
      // Write your code here
  if(exp<0) return -1;
        if(exp==0) return 1;
        return base* power(base, exp-1);
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Form a new number

Write a recursive function evenDigits that accepts an integer parameter n and returns a new integer containing only the even digits from n, in the same order. If n does not contain any even digits, return 0.
For example, the call of evenDigits(8342116) should return 8426 and the call of evenDigits(35179) should return 0.

Solution:

```
int evenDigits(int n)
{
  if(n==0) return;
    int rem=n%10;
    int small= evenDigits(n/10);
    if(rem%2==0) return small*10+rem;
    else return small;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Binary equivalent using recursion

Write a recursive function **decimalToBinary** that accepts an integer as a parameter and returns an integer whose digits look like that number's representation in binary (base-2). For example, the call of decimalToBinary(43) should return 101011.
Constraints: Do not use a string in your solution. Also do not use any built-in base conversion functions from the system libraries. solve the problem recursively.
Sample Input :
1 // no. of testcases
43
Sample Output :
101011

Solution:

```
int decimalToBinary(int n)
{
  if(n==0) return 0;
   return (n%2)+10*decimalToBinary(n/2);
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Greatest common divisor using recursion

Write a recursive function **gcd** that accepts two positive non-zero integer parameters *i* and *j* and returns the greatest common divisor of these numbers.

Sample Input

2 // Test Cases
30 18 // i j (testcase 1)
11 17 // i j (testcase 2)þÿ

Sample Output

6
1

Constraints:. Solve the problem recursively.

Solution:

```
int gcd(int i, int j)
{
  if(j==0) return i;
    return gcd(j,i%j);
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Sort an array using merge sort

Given an array of N integers, sort them in ascending order using merge sort (a divide and conquer approach). Merge sort cut the array in two halves, calls itself on these halves and then merge the two sorted halves recursively to make the whole array sorted. Write the two functions **mergeSort()** and **merge(),** in which the mergeSort() divides array in half and calls it recursively and merge() will merge the two sorted arrays given as parameters.

Input Format

First line contains the number of elements
Second line contains the elements of array separated by space.

Output Format

Print the elements of sorted array in ascending order.

Constraints

1 <= N <= 10^5
-(10^9) <= arr[i] <= 10^9

Sample Input

7
1 3 5 7 2 4 9

Sample Output

1 2 3 4 5 7 9

Solution:

```
class Result {
  // Merges two subarrays of arr[].
  // First subarray is arr[l..m] and Second subarray is arr[m+1..r]
  void merge(int array[], int l, int m, int r) {
int temp[]= new int[r-l+1];
      int i=l, j=m+1, k=0;
      while(i<=m && j<=r){
         if(array[i]>array[j]){
            temp[k++]=array[j++];
         }
         else{
             temp[k++]=array[i++];
         }
      }
      while(i<=m){
          temp[k++]=array[i++];
      }
      while(j<=r){
          temp[k++]=array[j++];
      }
      for(int p=0;p<temp.length;p++){
         array[l+p]=temp[p];
      }
  }
  /* l is for left index and r is right index of the sub-array of arr to be sorted */
  void mergeSort(int array[], int l, int r) {
if(l>=r){
    return;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
        int mid=(l+r)/2;
        mergeSort(array,l ,mid);
          mergeSort(array,mid+1 ,r);
        merge(array, l, mid, r);
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Sort an array using quick sort

Given an array of N integers, sort them in ascending order using quick sort (a divide and conquer approach). It picks an element as pivot and partitions the given array around the picked pivot element recursively. Write the two functions **quickSort()** and **partition(),** in which the quickSort() choose an pivot using partition() function and calls itself on the two parts of array before and after the pivot element.

Input Format

First line contains the number of elements
Second line contains the elements of array separated by space.

Output Format

Print the elements of sorted array in ascending order.

Constraints

1 <= N <= 10^5
-(10^9) <= arr[i] <= 10^9

Sample Input

7
1 3 5 7 2 4 9

Sample Output

1 2 3 4 5 7 9

Solution:

```
class Result {
  /* This function picks an element as pivot, places the pivot element at its correct position in sorted array, and places all smaller (smaller than pivot)
   to left of pivot and all greater elements to right of pivot */
  int partition (int array[], int low, int high) {
int pivot=array[high];
      int i=low-1;
    for(int j=low;j<high;j++){
       if(array[j]<pivot){
          i++;
          int temp=array[i];
          array[i]=array[j];
          array[j]=temp;
       }
    }
      int temp=array[i+1];
      array[i+1]=array[high];
      array[high]=temp;
      return i+1;
  }
  /* low is for left index and high is right index of the sub-array of arr to be sorted */
  void quickSort(int array[], int low, int high) {
      if(low>high) return;
int p=partition(array, low, high);
      quickSort(array, low, p-1);
      quickSort(array, p+1, high);
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Count the set bits of a number

Every number is represented in binary form in memory. A binary form consists of bits, which can take only two values i.e. 0 or 1. Given an integer number you have to count number of set bits in its binary representation.
Complete the function below which takes a number as argument and return the number of set bits i.e. bits with value 1.

Input Format

First line contains total number of Test Cases. Following T lines will contain one number each for test cases

Output Format

Number of Set Bits in each number each in a new line

Sample Input

3
4
5
14

Sample Output

1
2
3

Explanation:

4 is 0100
5 is 0101
14 is 1110

Solution:

```
int countBits(int num)
{
  int count=0;
    while(num>0){
        count+=num&1;
        num>>=1;
    }
    return count;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Toggle the number except kth bit

You are given a number N and another number k. Print the number after toggling all bits of N except the kth bit (0 based indexing on 64-bit machine where integer takes 4 bytes).

Input Format:

First line contains total number of Test Cases. Following T lines will contains two numbers (N, k) seperated by space each for test case.

Output Format:

Toggled number for each test case in new lines in decimal format.

Sample Input:

2
4 2
5 1

Sample Output:

-1
-8

Explanation:

In 8-bits, 4 is 0000 0100, so toggling all bits except bit no. 2, it becomes 1111 1111. which is -1
In 8-bits, 5 is 0000 0101, so toggling all bits except bit no. 1, it becomes 1111 1000. which is -8

Solution:

```
int toggleExceptKthBit(int n,int k){
  // Write your code here
    return ~n ^ (1<<k);
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find the odd man out

Given an array containing n numbers, in which, all the numbers are present even number of times except for one number which is occurring odd number of times. Find this odd man out in linear time i.e. without making double scan of the array.
Complete the ***findOddMan()*** function which accepts an array of integers, *arr as parameter and* must return an integer denoting the element in array occurring odd number of times.

Your solution must be optimized otherwise you may get an error "Time Limit Exceeded".
þÿ
Input Format:
First line contains an odd number ‘n’ i.e. total number of elements in the array. Then N lines follow containing the elements of array.
Output Format:
Print the number which occurs odd number of times.
Constraints:
1 <= n <= 10^5
-(10^9) <= arr[i] <= 10^9
Sample Input 1
7 // n
3
2
1
2
3
1
1
Sample Output 1
1
Explanation 1
1 is present three times whereas rest elements 2 and 3 are present twice.
Sample Input 2
9 // n
2
3
2
1
2
3
1
1
1
Sample Output 2
2
Explanation 2
2 is present three times, 1 is present at four positions and 3 is present at two positions.

Solution:

```
class Result {
  public static int findOddMan(int n, int arr[]) {
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    // Write your code here
int ans=0;
      for(int i=0;i<n;i++){
         ans^=arr[i];
      }
      return ans;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find the two strange elements

Given an array containing n elements, in which, all the elements are present even number of times except for two elements which occurs odd number of times. Find these two strange elements in ascending order in linear time with constant space complexity.

Input:

First line contains an even number ‘n’ i.e. total number of elements in the array. Second line contains n space separated elements of the array

Output:

Print the strange numbers separated by space in ascending order.

Constraints:

n < 10^8

Sample Input:

4
3 1 2 1

Sample Output:

2 3

Explanation:

1 is occurring two times, whereas 2 and 3 occurs only once.

Solution:

```
class Result{
  static void printStrangeElements(int[] arr, int n) {
    // Write your code here
      int ans=0;
      for(int i=0; i<n; i++){
          ans^=arr[i];
      }
      int bit= ans & -ans;
      int a=0, b=0;
      for(int i=0;i<n;i++){
      if ((arr[i] & bit) != 0){
          a^=arr[i];
      }
      else{
          b^=arr[i];
      }
  }
      if(a<b){
          System.out.println(a+" "+b);
      }
      else{
          System.out.println(b+" "+a);
      }
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Saving the Earth with Binary Fever

The Earth is under a threat from the outer space and the superheroes are busy in their personal life. Meanwhile, you have to catch the enemy and destroy him. Enemy is **N** steps above from you (assume yourself on first level i.e. level 0). Due to your binary fever, you can jump only in power of 2 i.e. 1, 2, 4, 8, 16 etc. So to defeat the enemy as quickly as possible, what is the minimum number of jumps you have to make?
Example: N = 25
Now, to reach the level 25 as quickly as possible, you can first make a jump of 1 unit, then of 8 units and finally a jump of 16 units. Therefore, a minimum of **3** jumps are required. Note that, jumps need not to be in increasing sequence only i.e. if you want, you can jump 8 units first, then 1 unit and finally 16 units.

Your solution must be optimized otherwise you may get an error "Time Limit Exceeded".

Input Format
The first line contains the number of test cases T. Next each line contains a number N which is the level of enemy.
Constraints:
1 <= T <= 10
0 <= N <= 2^53
Output Format
For each test case, print the minimum number of jumps shall be made by you to save the Earth, in a new line.
Sample Input:
2 // Test Cases
3 // N (testcase 1)
7 // N (testcase 2)
Sample Output:
2
3
Explanation:
Level 3 can be achieved by making jumps of 1 and 2, so total 2 jumps required.
Level 7 can be achieved by making jumps of 1, 2 and 4, so total 3 jumps required.

Solution:

```
class Result {
  // Return the minimum number of jumps
  static int getMinJumps(long num) {
    // Write your code here
int count=0;
      while(num>0){
         count+=num&1;
         num>>=1;
      }
      return count;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Print message using class

Write the function dispMsg() to display the message "Code Quotient - Get better at coding"

Solution:

```
// Write a method named dispMsg() her
public void dispMsg(){
   System.out.println("Code Quotient - Get better at coding");
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Class with get and set functions

Write the get and set methods for the class Triangle as below

```
class Triangle
{
 int base, height;
 void setbaseheight(int b, int h);
 float area();
 int getBase();
 int getHeight();þÿ
}
```

Solution:

```
void setbaseheight(int b, int h)
 {
   this.base=b;
     this.height=h;
 }
 float area()
 {
   return 0.5f*base*height;
 }
 int getBase()
 {
  return base;
 }
 int getHeight()
 {
  return height;
 }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Count of all digits of a number

Write a function to count and print the number of digits in a positive integer.
Complete the function **countDigits()** that accepts a number and returns the number of digits in it.

Input Format

The first line consists of number of testcases, T
Next T lines contains an integer whose numbers of digits is to be found

Output Format

For each test case print the number of digit in that number in a new line

Sample Input

1
12345

Sample Output

5

Solution:

```
/*
 * Complete the function countDigits
 * @params
 *   number -> integer , whose digits are to be calculated
 * @returns
 *   integer, number of digits in the number
 */
public int countDigits(int number){
  int count=0;
    while(number>0){
        count++;
        number/=10;
    }
    return count;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Calculate amount using compound interest

Write a program to calculate the amount using compound interest .

Input Format

Each test case contains three floating numbers denoting principle amount, rate and time.

Output Format

Print the amount upto 1(one) decimal point as shown below.

Sample Input

1000 5 2

Sample output

1102.5

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name
class Main{
    public static void main(String[] args)
    {
        // Write your code here
        Scanner sc = new Scanner (System.in);
        float p= sc.nextFloat();
          float r= sc.nextFloat();
          float t= sc.nextFloat();
        float result=(float)(p*Math.pow(1+r/100,t));
        System.out.printf("%.1f",result);
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Convert given number of seconds to days, hours, minutes and second

Given are number of seconds as an input , the task is to convert the given number of seconds to days, hours, minutes and seconds.
Complete the given functions to convert the given number of seconds to days, hours, minutes and seconds.

Input Format

The first line contains an integer, T , denoting the number of testcases
Then T lines follow, each consisting of an integer, the number of seconds

Output Format

For each testcase, print the number of days,hours,minutes,seconds.

Sample Input

1
128

Sample Output

0,0,2,8

Solution:

```
int getDays(int sec)
{
  return sec/86400;
}
int getHours(int sec)
{
  sec%=86400;
    return sec/3600;
}
int getMinutes(int sec)
{
  sec%=86400;
    sec%=3600;
    return sec/60;
}
int getSeconds(int sec)
{
  return sec%60;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Class - Rectangle

Write a class called **Rectangle** that represents a rectangular two-dimensional region. Your **Rectangle** objects should have the following methods:
public Rectangle(int x, int y, int width, int height)
Constructs a new rectangle whose top-left corner is specified by the given coordinates and with the given width and height.
public int getHeight()
Returns this rectangle's height.
public int getWidth()
Returns this rectangle's width.
public int getX()
Returns this rectangle's x-coordinate.
public int getY()
Returns this rectangle's y-coordinate.
public String toString()
Returns a string representation of this rectangle, such as ***"Rectangle[x=1,y=2,width=3,height=4]".***

Solution:

```
import java.util.Scanner;
// Other imports go here// Do NOT change the class name
class Rectangle
{
 int x;
    int y;
    int width;
    int height;
    Rectangle(int x, int y, int width, int height){
        this.x=x;
        this.y=y;
        this.width=width;
        this.height=height;
    }
    public int getHeight(){
        return height;
    }
    public int getWidth(){
        return width;
    }
    public int getX(){
        return x;
    }
    public int getY(){
        return y;
    }
    public String toString(){
        return "Rectangle[x="+x+",y="+y+",width="+width+",height="+height+"]";
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Class - TimeSpan

Define a class named TimeSpan. A TimeSpan object stores a span of time in hours and minutes (for example, the time span between 5:00am and 7:45am is 2 hours, 45 minutes). Each TimeSpan object should have the following public methods:
TimeSpan(hours, minutes)
Constructs a TimeSpan object storing the given time span of hours and minutes.
getHours()
Returns the number of hours in this time span.
getMinutes()
Returns the number of minutes in this time span, between 0 and 59.
add(hours, minutes)
Adds the given amount of time to the span. For example, (3 hours, 15 minutes) + (2 hour, 40 minutes) = (5 hours, 55 minutes). Assume that the parameters are valid: the hours are non-negative, and the minutes are between 0 and 59.
add(TimeSpan tp)
Adds the given amount of time (stored as a time span) to the current time span.
getTotalHours()
Returns the total time in this time span as the real number of hours, such as 9.75 for (9 hours, 45 min).
toString()
Returns a string representation of the time span of hours and minutes, such as "5 Hours, 38 Minutes".
Explanation:
The minutes should always be reported as being in the range of 0 to 59. That means that you may have to "carry" 60 minutes into a full hour.

Solution:

```
// Do NOT change the class name
class TimeSpan
{
  // Write your code here
int hours;
    int mins;
  public  TimeSpan(int hours,int mins){
      this.hours=hours;
      this.mins=mins;
  }
    public int getHours(){
        return hours;
    }
    public int getMinutes(){
        return mins;
    }
    void add(int hours, int mins){
        this.hours+=hours;
        this.mins+=mins;
        this.hours+=this.mins/60;
        this.mins=this.mins%60;
    }
    void add(TimeSpan tp){
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
        add(tp.hours, tp.mins);
    }
    double getTotalHours(){
        return hours+mins/60.0;
    }
    public String toString(){
        return hours+" Hours, "+mins+" Minutes";
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Class - Date

Write a class named **Date** that remembers information about a month and day. Ignore leap years and don't store the year in your object. You must include the following public methods:
Member-name Description
Date(int m, int d) constructs a new date representing the given month and day
daysInMonth() returns the number of days in the month stored by your date object
getDay() returns the day
getMonth() returns the month
nextDay() advances the Date to the next day, wrapping to the next month and/or year if necessary
toString() returns a string representation e.g. If date is 27 June then return "6/27"
absoluteDay() return the absolute day of a particular date e.g. January 1 is 1, February 1 is 32, March 1 is 60 and December 31 is 365.
You should define the entire class including the class heading, the private fields, and the declarations and definitions of all the public methods and constructor.

Solution:

```
class Date
{
	// Write your code here
int month;
   int day;
   Date(int month, int day){
      this.month=month;
      this.day=day;
   }
   int daysInMonth(){
      switch(month){
         case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
         case 2:
            return 28;
         default:
            return 30;
      }
   }
   public int getDay(){
      return day;
   }
   public int getMonth(){
      return month;
   }
   public void nextDay(){
      day++;
      if(day>daysInMonth()){
         day=1;
         month++;
         if(month>12){
            month=1;
         }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
        }
    }
public int absoluteDay(){
    int total=day;
    for(int m=1; m<month; m++){
        switch(m){
                case 1: case 3: case 5: case 7: case 8: case 10: case 12:
                total+= 31;
                break;
            case 2:
                total+= 28;
                break;
            default:
               total+= 30;
                break;
        }
    }
    return total;
}
    public String toString(){
        return month+ "/" + day;
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Sum of Range

Given 2 numbers min & max, the task is to find the sum of the elements present in the range [min,max].
Complete the function **sumOfRange()** that accepts two integer parameters *min* and *max* and returns the sum of the integers from *min* through *max* inclusive.
For example, the call of **sumOfRange(3, 7)** returns 25 (3 + 4 + 5 + 6 + 7) . If *min* is greater than *max*, return 0.

Input Format

The first line of input contains the number of testcases, T
Then T lines follow, each line contains 2 integers, min & max, separated by a space

Output Format

For each testcase, print the sum of elements present in the range [min,max]

Sample Input

2
3 7
0 5

Sample Output

25
15

þÿExplanation:

For the first case , the sum of range [3,7] is (3+4+5+6+7) = 25
For the second case , the sum of range [0,5] is (0+1+2+3+4+5) = 15

Solution:

```
int sumOfRange(int min, int max){
  // Write your code here
    int sum=0;
    for(int i=min; i<=max;i++){
        sum+=i;
    }
    return sum;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Verify Prime Number

Given an integer n, the task is to check whether that the given integer is a prime number or not
Complete the function **verifyPrime()** given in the editor, that accepts an integer parameter n and returns true/1 if number is prime and false/0 if number is not prime.

Your solution must be optimized otherwise you may get an error "Time Limit Exceeded".
þÿ
Input Format:
The first line of input contains the number of test cases , T
Then T lines follow, each line contains an integer, n which is to be checked whether it is prime or not
Constraints:
1 "d T "d 100
0 "d n "d 10^9
Output Format:
Print 'PRIME', if the number is a prime number else 'NOT PRIME' for each testcase in a new line.
Sample Input
2 // No. of testcases
3
4
Sample Output
PRIME
NOT PRIME

Solution:

```
/*
 * Complete the function 'verifyPrime'
 * @params
 *   n ->number which is to be checked from primality test
 *
 * @return
 *   true if the number is a prime number else false
 */
static boolean verifyPrime(int n) {
  // Write your code here
    if(n<2){
        return false;
    }
    for(int i=2;i*i<=n;i++){
        if(n%i==0){
            return false;
        }
    }
    return true;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Java : toString() method - 1

A BankAccount class is defined as below: -

```
class BankAccount {
   private String name;
   private double balance;

   // your code goes here
}
```

add the following method to it:

public String toString()

Your method should return a string that contains the account's name and balance separated by a comma and space. For example, if an account object named acc1 has the name "Amit" and a balance of 120.4, the call of acc1.toString() should return:

Amit, 120.4

There are some special cases you should handle. If the balance is negative, put the message as below. Also, always display the paise as a one-digit number. For example, if the same object had a balance of -17.5, your method should return:

Amit, -17.5 (NEGATIVE)

Solution:

```
public String toString()
{
  String result=name+", "+balance;
    if(balance<0){
        result+=" (NEGATIVE)";
    }
    return result;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Java : class - BankFee

A BankAccount class is defined as below: -

```
class BankAccount {
  private String name;
  private double balance;
  private int transactions;

  // Constructs a BankAccount object with the given name, and 0 balance and transactions.
  public BankAccount(String name)

  // returns the field values
  public double getBalance()
  public String getName()
  public String getTransactions()

  // Adds the amount to the balance if it is between 0-500 also counts as 1 transaction.
  public void deposit(double amount)

  // Subtracts the amount from the balance if the user has enough money also counts as 1 transaction.
  public void withdraw(double amount)
}
```

add the following method to it:

public boolean bankFees(double fee)

The bankFees method accepts a fee amount (a double) as a parameter, and applies that fee to the user's past transactions. The fee is applied once for the first transaction, twice for the second transaction, three times for the third, and so on. These fees are subtracted out from the user's overall balance. If the user's balance is large enough to afford all of the fees with greater than 0.00 remaining, the method returns true. If the balance cannot afford all of the fees or has no money left, the balance is left as 0.0 and the method returns false. For example, given the following BankAccount object:

```
BankAccount o1 = new BankAccount("Amit");
o1.deposit(10.00);
o1.deposit(30.00);
o1.deposit(20.00);
o1.deposit(80.00);
```

The account at that point has a balance of 140.00.

If the following call was made:

```
o1.bankFees(5.00);
```

Then the account would be deducted 5 + 10 + 15 + 20 for the four transactions, leaving a final balance of 90.00. The method would return true.

However, If the call was made as below,

```
o1.bankFees(20.00);
```

Then the account would be deducted 20 + 40 + 60 + 80 for the four transactions, leaving a final balance of -60.00. The method would return false.

þÿSample Input

```
Arun // Name of account holder
10 // deposit-1
30 // deposit-2
20 // deposit-3
80 // deposit-4
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

5 // bank fee

Sample Output

true

Solution:

```
public boolean bankFees(double fees)
{
  double chrg=0;
    for(int i=0;i<=transactions;i++){
        chrg+=fees*i;
    }
    if(balance-chrg > 0){
        return true;
    }
    else{
        balance=0.0;
        return false;
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Class - Circle

Write a class of objects named Circle that remembers information about a circle. You must include the following public members.
You can use the constant named Math.PI storing the value of < Â &÷Vv†Ç’ 2ã C S’à
Name Description
Circle(double r) constructs a new circle with the given radius as a real number
area() returns the area occupied by the circle (upto 2 decimal places)
circumference() returns the distance around the circle (upto 2 decimal places)
getRadius() returns the radius as a real number (upto 1 decimal places)
toString() returns a string representation such as "Circle{radius=2.5}"
You should define the entire class including the class heading, the private fields, and the declarations and definitions of all the public methods and constructor.

Solution:

```
class Circle
{
  double radius;
    Circle(double radius){
       this.radius=radius;
    }
    public double area(){
       return Double.parseDouble(String.format("%.2f",Math.PI * radius * radius));
    }
     public double circumference(){
       return Double.parseDouble(String.format("%.2f",2 * Math.PI * radius));
    }
    public double getRadius(){
       return Double.parseDouble(String.format("%.1f",radius));
    }
    public String toString(){
       return "Circle{radius="+radius+"}";
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Class - TollBooth

Imagine a tollbooth at a bridge. Cars passing by the booth are expected to pay 50 Rupees toll. Mostly they do, but sometimes a car goes by without paying. The tollbooth keeps track of the number of cars that have gone by, and of the total amount of money collected.
Model this tollbooth with a class called TollBooth. The two data items are to hold the total number of cars, and to hold the total amount of money collected.
A constructor initializes both of these to 0.A member function called payingCar() increments the car total and adds 50/- to the cash total.Another function, called nopayCar(), increments the car total but adds nothing to the cash total.Finally, a member function called display() displays the two totals.
Make appropriate member functions const.
The main function should allow the user to press 'p' to count a paying car, and 'n' to count a nonpaying car. Pushing the 'q' key should cause the program to print out the total cars and total cash and then exit.

Sample Input

p
p
p
n
p
n
p
q

Sample Output

Total Cash : 250/-
Total Cars : 7

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name
class TollBooth{
    int cars;
    int cash;
    TollBooth(){
        this.cars=0;
        this.cash=0;
    }
    void payingCar(){
        cars++;
        cash+=50;
    }
    void nopayCar(){
        cars++;
    }
    void display(){
        System.out.println("Total Cash : "+cash+"/-");
        System.out.println("Total Cars : "+ cars);
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
class Main{
    public static void main(String[] args)
    {
        // Write your code here
         Scanner sc = new Scanner(System.in);
        TollBooth tb = new TollBooth();
        while(true)
        {
            char ch = sc.next().charAt(0);
            if(ch == 'p')
            {
                tb.payingCar();
            }
            else if(ch == 'n')
            {
                tb.nopayCar();
            }
            else if(ch == 'q')
            {
                tb.display();
                break;
            }
        }
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Functions : Addition of numbers

Write the overloaded ***sum()*** functions for addition of 2, 3 or 4 numbers passed to it.

Sample Input1

2
4 5

Sample Output1

9

Sample Input2

3
4 5 8

Sample Output2

17

Solution:

```
// Write the sum() function here.
int sum(int a, int b){
   return a+b;
}
int sum(int a, int b, int c){
   return a+b+c;
}
int sum(int a, int b, int c, int d){
   return a+b+c+d;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Functions : Display of Strings

Write the overloaded display function to display either 1 string in one line or 2 strings in one line with a hyphen '-' in between.

Sample Input1

1
CodeQuotient

Sample Output1

CodeQuotient

Sample Input2

2
CodeQuotient
coding

Sample Output2

CodeQuotient-coding

Solution:

```
// // Write overloaded static void display(String a) and static void display(String a, String b) functions
static void display(String a){
   System.out.println("CodeQuotient");
}
static void display(String a, String b){
     System.out.println("CodeQuotient-coding");
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: class - Box

Design a class named *Box* whose dimensions are integers and private to the class.
The dimensions are labelled: length, breadth and height.
The default constructor of the class should initialize them to 0 (ZERO).The parameterized constructor *Box(int length, int breadth, int height)* should initialize *them* to length, breadth and height respectively.The copy constructor *Box(Box* b1) should set them to b1's length, breadth and height respectively.
Apart from the above, the class should have functions:
int getLength() - Return box's length*int getBreadth()* - Return box's breadth*int getHeight()* - Return box's height*long long CalculateVolume()* - Return the volume of the box

Solution:

```
class Box
{
  int length;
    int breadth;
    int height;
    Box(){
        this.length=0;
        this.breadth=0;
        this.height=0;
    }
  Box(int length, int breadth, int height) {
      this.length=length;
        this.breadth=breadth;
        this.height=height;
  }
    Box (Box b1){
       this.length=b1.length;
        this.breadth=b1.breadth;
        this.height=b1.height;
    }
    int getLength(){
        return length;
    }
    int getBreadth(){
        return breadth;
    }
    int getHeight(){
        return height;
    }
long CalculateVolume(){
        return (long)length*breadth*height;
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Password too short Exception

Write a program to accept user name and password from the user.
If the password has less than 6 characters then throw an exception as character 's'If the password does not contain a digit then throw an exception as character 'd'.
For handling exception implement necessary catch blocks and print the messages accordingly.

Sample Input1
Arun
abcd123
SampleOutput1
Correct
Sample Input2
Arun
abcdefgh
SampleOutput2
No digit!
Sample Input3
Arun
abc1
SampleOutput3
Too short!

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name
class Main
{
  public static void main(String[] args)
  {
    // Write your code here
      Scanner sc= new Scanner (System.in);
      String user=sc.nextLine();
      String pass=sc.nextLine();
      if(pass.length()<6){
          System.out.println("Too short!");
          return;
      }
      boolean digit=false;
      for(int i=0; i<pass.length() ;i++){
          if(Character.isDigit(pass.charAt(i))){
              digit=true;
              break;
          }
      }
      if(digit){
           System.out.println("Correct");
      }
      else{
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            System.out.println("No digit!");
        }
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Valid email address

Check an email-id for following exceptions
if id does not contain '@' **OR**if id does not contain '.' (dot) **OR**if no '.' (dot) appears after '@'
catch the exception and print "**Invalid**" or "**Valid**" accordingly.

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name
class Main
{
  public static void main(String[] args)
  {
    String id;
    Scanner s1=new Scanner(System.in);
    id=s1.nextLine();
      if(!id.contains("@") || !id.contains(".")){
          System.out.println("Invalid");
          return;
      }
      int dot_idx=id.indexOf(".");
      int at_idx=id.indexOf("@");
      if(dot_idx < at_idx){
            System.out.println("Invalid");
      }
      else{
            System.out.println("Valid");
      }
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Valid index of array

Write a program to check whether the indexes given are valid index are not. Print "**Out of Bounds**" if any attempt is made to refer to an element whose index is beyond the array size , else print the element present at the index.

Input Format

The first line of input contains n , denoting the size of array.
The second line contains n spaced integers , denoting elements of array
The third line contains q, denoting number of queries
Next q lines contains an integer, index, which is to be tested.

Output Format

For each query print the value at index if the index is a valid index , else print "Out of Bounds".

Sample Input

```
10      // Size of array
1 2 3 4 5 6 7 8 9 10  // array elements
2 // No. Of Queries
4    // index to be retrieved
13    // index to be retrieved
```

Sample Output

```
5
Out of Bounds
```

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name
class Main{
   public static void main(String[] args)
   {
      // Write your code here
      Scanner sc= new Scanner(System.in);
      int n=sc.nextInt();
      int arr[]=new int[n];
      for(int i=0;i<n;i++){
         arr[i]=sc.nextInt();
      }
      int q=sc.nextInt();
      for(int i=0;i<q;i++){
         int idx=sc.nextInt();
         try{
             System.out.println(arr[idx]);
         }
         catch(Exception e){
            System.out.println("Out of Bounds");
         }
      }
   }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Java: Inheritance - MinMaxAccount

A class called **BankAccount** with two integer members as ***number, balance*** and many methods has been defined:

Method/Constructor	Description
public BankAccount(int number, int bal)	constructs a BankAccount object with number and opening balance provided.
public void withdraw(int amt)	records the given withdrawl
public void deposit(int amt)	records the given deposit
public int getBalance()	returns current balance in INR

Design a new class **MinMaxAccount** whose instances can be used in place of a BankAccount object but include new behavior of remembering the minimum and maximum balances ever recorded for the account.

You should provide the same methods as the superclass, as well as the following new behavior:

Method/Constructor	Description
public MinMaxAccount(int number, int bal)	constructs a MinMaxAccount object using information in the Startup object s
public int getMin()	returns minimum balance in INR
public int getMax()	returns maximum balance in INR

Assume that only the debit and credit methods change an account's balance.

Sample Input

```
1234  // account number
500   // opening balance
1500  // deposit
500   // deposit
1800  // withdrawl
4500  // deposit
4600  // withdraw
```

Sample Output

```
500   // Opening balance
2000  // balance after deposit 1500
2500  // balance after deposit 500
700   // balance after withdraw 1800
5200  // balance after deposit 4500
600   // balance after withdraw 4600
500   // Minimum Balance
5200  // Maximum Balance
```

Solution:

```
class MinMaxAccount extends BankAccount
{
   // Complete the class definition according to requirement
int min;
int max;
MinMaxAccount(int number, int bal){
super(number, bal);
this.min=bal;
this.max=bal;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
public void deposit(int amt){
super.deposit(amt);
int bal=getBalance();
if(bal>max){
max=bal;
}
}
public void withdraw(int amt){
super.withdraw(amt);
int bal=getBalance();
if(bal<min){
min=bal;
}
}
public int getMin(){
return min;
}
public int getMax(){
return max;
}
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Java : Inheritance - Marketer

Given the below class Employee, you have to write a new class Marketers.

```
class Employee {
   private int baseHours;
   private int baseSalary;
   private int baseVacationDays;
   private String baseVacationForm;

   public int getHours();
   public int getSalary();
   public int getVacationDays()
   public String getVacationForm()

   public final void setBaseHours(int hours);
   public final void setBaseSalary(int salary);
   public final void setBaseVacationDays(int days);
   public final void setBaseVacationForm(String form);
}
```

Marketers make INR 5,000 as salary, and have an additional method called "message" that prints "CodeQuotient - Get better at coding". Make sure to interact with the Employee superclass as appropriate.

Solution:

```
// Create a class Marketers to support above class.
class Marketers extends Employee{
    public int getSalary(){
        return 5000;
    }
   public void message(){
       System.out.println("CodeQuotient - Get better at coding");
   }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Inheritance : Actors

Design a class Person and two sub classes Actor and Actress. Every Actor will have a name, color, number_of_eyes & debut_year associated with him/her.
Supply each with a constructor that sets the Person data û elds as described with given values, and a method named **toString()** that returns a string that contains a complete description of the Actor.
The constructor should be in the following format
Actor(name,color,number_of_eyes,debut_year)
And the same for class **Actress.**

Sample input

Amitabh
BROWN
2
1965
Hema
White
2
1975

Sample output

The person Amitabh is an Actor. He is BROWN in color, has 2 eyes and debut in 1965
The person Hema is an Actress. She is White in color, has 2 eyes and debut in 1975

Solution:

```
class Person
{
  String name;
    String color;
    int eyes;
    int year;
    Person(String name, String color, int eyes, int year){
        this.name=name;
        this.color=color;
        this.eyes=eyes;
        this.year=year;
    }
}
class Actor extends Person
{
Actor(String name, String color, int eyes, int year){
super(name, color, eyes, year);
}
public String toString(){
return "The person "+name+" is an Actor. He is "+color+" in color, has "+eyes+" eyes and
debut in "+year;
}
}
class Actress extends Person
{
Actress(String name, String color, int eyes, int year){
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
super(name, color, eyes, year);
}
public String toString(){
return "The person "+name+" is an Actress. She is "+color+" in color, has "+eyes+" eyes and
debut in "+year;
}
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Java : Inheritance - 3

Suppose that the following two classes have been declared:

```
class A {
   public void m1() {
      System.out.println("Coding");
   }
   public void m2() {
      System.out.println("Quotient");
   }
    public String toString() {
      return "cq";
   }
}

class B extends A {
   public void m1() {
      System.out.println("Code");
   }
   public String toString() {
      return (super.toString() + super.toString());
   }
}
```

Write a class **C** which extends class **B** above & whose methods have the behavior below. Don't just print/return the output; whenever possible, use inheritance to reuse behavior from the superclass.

Method	Output/Return
m1	Code
m2	Code Quotient
toString	newcqcq

Solution:

```
// Write class C here.
class C extends B{
   public void m2(){
      m1();
      super.m2();
   }
    public String toString(){
      return "new"+super.toString();
   }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Inheritance : CalculateBill

Design a solution that has following features:
Generate bill on the basis of Item price and quantity i.e. **Bill = price of item * quantity**
Calculates cash from notes of Rs 2000, Rs 500, Rs 100, Rs 50, and Rs 10.Match cash against bill and display “Clear” message if no balance was there otherwise print the amount needs to pay.

Develop two classes Bill and Cash, where Cash inherits Bill. Sample input and output are shown below:
Sample Input 1:
1000 //item_price
100 //qty
4 //notes of 2000
0 //notes of 500
0 //notes of 100
0 //notes of 50
10 //notes of 10
Sample Output 1:
Need to pay: 91900
Constraints: notes should be of Rs 2000, Rs 500, Rs 100, Rs 50, and Rs 10.
Formula Used: Bill = price of item * quantity
Solution:

```
class Bill
{
int price;
    int qty;
    int bill;
    void calculate(){
        bill=price*qty;
    }
}
class Cash extends Bill
{
int total;
int n2000, n500, n100, n50, n10;
 void get_cash()
 {
 Scanner sc=new Scanner (System.in);
 price=sc.nextInt();
 qty=sc.nextInt();
 calculate();
 n2000=sc.nextInt();
 n500=sc.nextInt();
 n100=sc.nextInt();
 n50=sc.nextInt();
 n10=sc.nextInt();
 total=n2000 * 2000 + n500 * 500 + n100 * 100 + n50 * 50 + n10 * 10;
    // Complete the input function, dont change the name.
 }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
 void display()
 {
 if(total<bill){
 System.out.println("Need to pay: "+(bill-total));
 }
 else{
  System.out.println("Clear");
 }
    // Complete the display function, dont change the name.
 }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Inheritance : BookCD

Imagine a publishing company that markets both Book and CD's of its works. A class Publication is created that stores the title (a string) and price (type int) of a publication. From this class derive two classes:
Book, which adds a page count and author, and a constructor having title, price, pages & writer as parameters.CD, which adds a playing time in minutes, and a constructor having title, price & length as parameters.
Each of these two classes should have a putdata() function to display its data as shown in sample output.

Sample Input

Programming-In-C // Title of Book
150 // Price of Book
1025 // Pages of Book
Schildt // Writer of Book
Rock-On // Title of CD
50 // Price of CD
185 // Length of CD

Sample Output

Book Title "Programming-In-C", written by "Schildt" has 1025 pages and of 150 rupees.
CD Title "Rock-On", is of 185 minutes length and of 50 rupees.

Solution:

```
class Book extends Publication
{
int pages;
String author;
Book(String title, int price, int pages, String author){
this.title=title;
this.price=price;
this.pages=pages;
this.author=author;
}
void putdata(){
System.out.println("Book Title \""+title+"\", written by \""+author+"\" has "+pages+" pages
and of "+price+" rupees.");
}
}
class CD extends Publication
{
int length;
CD(String title, int price, int length){
this.title=title;
this.price=price;
this.length=length;
}
void putdata(){
System.out.println("CD Title \""+title+"\", is of "+length+" minutes length and of "+price +"
rupees.");
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
}
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Inheritance : FilteredAccount

A cash processing company has a class called **Account** used to process transactions:
Method/Constructor Description
Account(int accno) **constructs an account using account number given**
boolean process(Transaction t) **processes the next transaction, returning true if transaction was approved, false otherwise**
Account objects interact with **Transaction** objects, which have many methods including:
int value() **returns the value of this transaction in rupees (could be negative, positive or zero)**
The company wishes to create a slight modification to the Account class that filters out zero-valued transactions. Design a new class called **FilteredAccount** whose instances can be used in place of an **Account** object but which include the extra behavior of not processing transactions with a value of 0. More specifically, the new class should indicate that a zero-valued transaction was approved but shouldn't call the process method in the **Account** class to process it. Your class should have a single constructor that accepts a parameter of type int, and it should include the following method:
int filtered() **returns the number of transactions filtered out; returns 0 if no transaction submitted**

(**Hint**: override the process() method and call Account class process() for non-zero value transaction, otherwise ignore it. Also put a counter for transactions and manipulate in process() method.)

Solution:

```
class FilteredAccount extends Account
{
   // Write your code here
   int count=0;
   FilteredAccount(int acc){
   super(acc);
   }
  public boolean process(Transaction t){
   if(t.value()==0){
   count++;
   return true;
   }
   else{
  return super.process(t);
   }
   }
   public int filtered(){
   return count;
   }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Inheritance : MemoryCalculator

A class Calculator that performs calculations on integers is defined as below: -

Member	Description
Calculator(int seed)	**constructs a Calculator with given seed for random numbers**
boolean isPrime(int n)	**returns true if n is prime**
int kthPrime(int k)	**returns the kth prime (assumes k >= 1)**
int fib(int n)	**returns the nth Fibonacci number (assumes n >= 1)**
int rand(int max)	**returns a random value between 0 and max**

The class correctly computes its results, but it does so inefficiently. In particular, it often computes the same value more than once. You are to implement a technique known as "memoizing" to speed up the computation of primes. The idea behind memoizing is to remember values that have been computed previously. For example, suppose that the value kthPrime(30) is requested 100 times. There is no reason to compute it 100 different times. Instead you can compute it once and store its value, so that the 99 calls after the first simply return the "memoized" value (the remembered value).

Define a new class called MemoryCalculator that can be used in place of a Calculator to speed up the prime computation. A MemoryCalculator object should behave just like a Calculator object except that it should guarantee that the value of kthPrime(k) is computed only once for any given value k. Your class should still rely on the Calculator class to compute each value for kthPrime(k). It is simply guaranteeing that the computation is not performed more than once for any particular value of k. The isPrime method calls kthPrime, so it does not need to be memoized. You do not need to memoize the Fibonacci computation. You should not make any assumptions about how large k might be or about the order in which the method is called with different values of k.

Your class should also provide the following public member functions that will allow a client to find out how many values have been directly computed versus how many calls have been handled through memoization.

Member	Description
MemoryCalculator(int seed)	**constructs a MemoryCalculator with given seed for random numbers**
int getComputeCount()	**returns number of values actually computed**
int getMemoCount()	**returns number of calls handled through memoization**

Sample Input

```
3  // seed for random numbers
3  // No. of time the kthPrime() function will be called
7  // Parameter for kthPrime() function
```

Sample Output

```
1 // Number of times kthPrime() is called
2 // Number of times kthPrime() result is fetched from memory
```

Solution:

```
class MemoryCalculator extends Calculator
{
   // Write your code here
   int mem[]=new int [10000];
int memory=0;
int compute=0;
MemoryCalculator(int seed){
super(seed);
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
public int kthPrime(int n){
if(mem[n]!=0){
memory++;
return mem[n];
}
else{
mem[n]=super.kthPrime(n);
compute++;
return mem[n];
}
}
public int getComputeCount() {
return compute;
}
public int getMemoCount(){
return memory;
}
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Java : Interfaces - 1

Create two classes "Car" and "Bike" from interface "Vehicle" as below:

```
interface Vehicle {
   void changeGear(int a);  // To set the gear to a.
   void speedUp(int a);      // To add a in speed.
   void applyBrakes(int a);  // To decrease a from speed.
}
```

The classes must have gear and speed class variables and also a toString method to return a string as "150kmph" if 150 is the current speed.

Solution:

```
// Create classes Car and Bike here as specified.
class Car implements Vehicle{
    int gear;
    int speed;
 public   void changeGear(int a){
        gear=a;
    }
 public     void speedUp(int a){
          speed+=a;
      }
  public   void applyBrakes(int a){
         speed-=a;
     }
    public String toString(){
        return speed+"kmph";
    }
}
// Create classes Car and Bike here as specified.
class Bike implements Vehicle{
    int gear;
    int speed;
  public  void changeGear(int a){
        gear=a;
    }
   public   void speedUp(int a){
         speed+=a;
      }
   public  void applyBrakes(int a){
         speed-=a;
     }
    public String toString(){
        return speed+"kmph";
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Class - Writer and Book

Design a class called Writer, which is designed to model a book's writer. It contains:
Three private instance variables: name(String), email(String), and gender(char of either 'm' or 'f');One constructor to initialize the name, email and gender with the given values;
public Writer(String name, String email, char gender) {.....}
public getters/setters: getName(), getEmail(), setEmail(), and getGender();
(There are no setters for name and gender, as these attributes cannot be changed.)
A toString() method that returns "Writer[name=?,email=?,gender=?]", e.g., "
Writer[name=Girdhar Gopal,email=girdhar@codequotient.com,gender=m]".

Also, design a class called Book to model a book written by *one* writer. It contains:
Four private instance variables: name(String), writer(of the class Writer you have just created, assume that a book has one and only one writer), price(double), and qty(int);Two constructors:
public Book (String name, Writer writer, double price) { ..... }
public Book (String name, Writer writer, double price, int qty) { ..... }
public methods getName(), getPrice(), setPrice(), getQty(), setQty().A toString() that returns "Book[name=?,Writer[name=?,email=?,gender=?],price=?,qty=?". You should reuse Writer's toString().

Solution:

```
class Writer
{
  String name;
    String email;
    char gender;
   public Writer(String name, String email, char gender){
       this.name=name;
       this.email=email;
       this.gender=gender;
   }
    public String getName(){
        return name;
    }
    public String getEmail(){
return email;
    }
    public void setEmail(String email){
       this.email=email;
    }
    public char getGender(){
       return gender;
    }
    public String toString(){
       return "Writer[name="+name+",email="+email+",gender="+gender+"]";
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
class Book
{
  String name;
    Writer writer;
    double price;
    int qty;
    public Book (String name, Writer writer, double price){
        this.name=name;
        this.writer=writer;
        this.price=price;
    }
    public Book (String name, Writer writer, double price, int qty){
         this.name=name;
        this.writer=writer;
        this.price=price;
        this.qty=qty;
    }
    public String getName(){
        return name;
    }
    public double getPrice(){
        return price;
    }
    public void setPrice(double price){
        this.price=price;
    }
    public int getQty(){
        return qty;
    }
    public void setQty(int qty){
        this.qty=qty;
    }
    public String toString(){
        return "Book[name="+name+","+writer.toString()+",price="+price+",qty="+qty;
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Class - Writer and Book - 2

Design a class called Writer, which is designed to model a book's writer. It contains:
Three private instance variables: name(String), email(String), and gender(char of either 'm' or 'f');One constructor to initialize the name, email and gender with the given values;
public Writer(String name, String email, char gender) {.....}
public getters/setters: getName(), getEmail(), setEmail(), and getGender();
(There are no setters for name and gender, as these attributes cannot be changed.)
A toString() method that returns "Writer[name=?,email=?,gender=?]", e.g., "
Writer[name=Girdhar Gopal,email=girdhar@codequotient.com,gender=m]".

Also, design a class called Book to model a book written by *one* writer. It contains:
Four private instance variables: name(String), writers(an array of the class Writer you have just created, assume that a book can have any number of writers), price(double), and qty(int);
Two constructors:
public Book (String name, Writer[] writer, double price) { ..... }
public Book (String name, Writer[] writer, double price, int qty) { ..... }
public methods getName(), getPrice(), setPrice(), getQty(), setQty().A toString() that returns "Book[name=?,Writers={Writer[name=?,email=?,gender=?],.....},price=?,qty=?". You should reuse Writer's toString().

Solution:

```
class Writer
{
  String name;
    String email;
    char gender;
    Writer(String name, String email, char gender){
        this.name=name;
        this.email=email;
        this.gender=gender;
    }
    public String getName(){
        return name;
    }
    public String getEmail(){
        return email;
    }
    public char getGender(){
        return gender;
    }
    public void setEmail(String email){
        this.email=email;
    }
    public String toString(){
        return "Writer[name="+name+",email="+email+",gender="+gender+"]";
    }
}
```

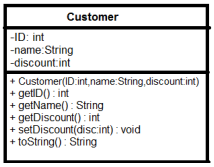

RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
class Book
{
 String name;
   Writer writers[];
   double price;
   int qty;
   public Book (String name, Writer[] writers, double price) {
   this.name=name;
      this.writers=writers;
      this.price=price;
   }
public Book (String name, Writer[] writers, double price, int qty) {
this.name=name;
   this.writers=writers;
   this.price=price;
   this.qty=qty;
}
      public String getName(){
         return name;
      }
   public double getPrice(){
      return price;
   }
   public void setPrice(double price){
      this.price=price;
   }
   public void setQty(int qty){
      this.qty=qty;
   }
   public int getQty(){
      return qty;
   }
   public String toString(){
      String res="Book[name="+name+",Writers={";
      for(int i=0;i<writers.length;i++){
         res+=writers[i].toString();
         if(i!=writers.length-1){
            res+=",";
         }
      }
      res+="},price="+price+",qty="+qty;
      return res;
   }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

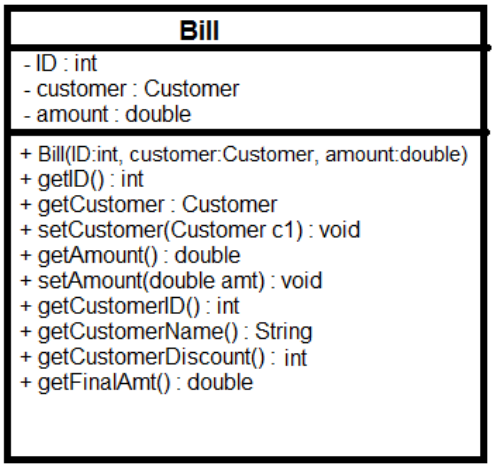
Aim: Class - Customer and Bill

Design a Customer class as per the below class diagram: -



NOTES:
discount is the %age of discount given to a Customer.
toString() method should print as "**name(ID)**".

Design a Bill class as shown in below class diagram: -



Solution:

```
class Customer
{
  int id;
    String name;
    int discount;
    Customer(int id, String name, int discount){
        this.id=id;
        this.name=name;
        this.discount=discount;
    }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    public int getID(){
        return id;
    }
    public String getName(){
        return name;
    }
    public int getDiscount(){
        return discount;
    }
    public void setDiscount(int discount){
        this.discount=discount;
    }
    public String toString(){
        return name+"("+id+")";
    }
}
class Bill
{
  int id;
    Customer customer;
    double amount;
    Bill(int id, Customer customer, double amount){
        this.id=id;
        this.customer=customer;
        this.amount=amount;
    }
    public int getID(){
        return id;
    }
    public Customer getCustomer(){
        return customer;
    }
    public void setCustomer(Customer c1){
        this.customer=c1;
    }
    public double getAmount(){
        return amount;
    }
    public void setAmount(double amt){
        this.amount=amt;
    }
    public int getCustomerID(){
        return customer.getID();
    }
    public String getCustomerName(){
        return customer.getName();
    }
    public int getCustomerDiscount(){
        return customer.getDiscount();
    }
    public double getFinalAmt(){
         return amount - (amount * customer.getDiscount() / 100.0);
```

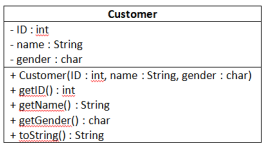

RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

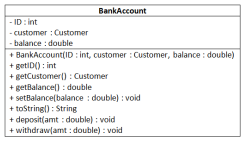
Aim: Class - Customer and BankAccount

Design a Customer class as per the below class diagram: -



In toString() method, you should print as below:
name(ID)

Design a BankAccount class as shown in below class diagram: -



- In toString() method, you should print as below:
  name(ID) balance=INR xxxx.xx
  balance will be rounded to 2 places and reuse Customer's toString().
- deposit() will add amt to balance.
- withdraw will subtract amt from balance if possible and print "Insufficient balance" otherwise

Solution:

```
class Customer
{
  int id;
    String name;
    char gender;
    Customer(int id, String name, char gender){
        this.id=id;
        this.name=name;
        this.gender=gender;
    }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    public int getID(){
        return id;
    }
    public String getName(){
        return name;
    }
    public char getGender(){
        return gender;
    }
    public String toString(){
        return name+"("+id+")";
    }
}
class BankAccount
{
  int id;
    Customer customer;
    double balance;
    BankAccount(int id, Customer customer, double balance){
        this.id=id;
        this.customer=customer;
        this.balance=balance;
    }
    public int getID(){
        return id;
    }
    public Customer getCustomer(){
        return customer;
    }
    public double getBalance(){
        return balance;
    }
    public void setBalance(double balance){
        this.balance=balance;
    }
    public String toString(){
        return customer.toString()+" balance=INR "+(String.format("%.2f",balance));
    }
    public void deposit(double amt){
        balance+=amt;
    }
    public void withdraw(double amt){
        if(balance>=amt){
            balance-=amt;
        }
        else{
            System.out.println("Insufficient balance");
        }
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Class - MovablePoint and MovableCircle

Let us have a set of objects with some common behaviors i.e. they could move up, down, left or right. The exact behaviors (such as how to move and how far to move) depend on the objects themselves.
One common way to model these common behaviors is to define an interface called Movable, with abstract methods moveUp(), moveDown(), moveLeft() and moveRight(). The classes that implement the Movable interface will provide actual implementation to these abstract methods. The code for the interface Movable is straight forward.

```
interface Movable
{
  public void moveUp();
  public void moveDown();
  public void moveLeft();
  public void moveRight();
}
```

Now write two concrete classes - MovablePoint and MovableCircle - that implement the Movable interface.
For the MovablePoint class, declare the instance variable x, y, xSpeed and ySpeed with package access (i.e., classes in the same package can access these variables directly).For the MovableCircle class, use a MovablePoint to represent its center (which contains four variable x, y, xSpeed and ySpeed). In other words, the MovableCircle composes a MovablePoint, and its radius.

```
class MovablePoint implements Movable
{
   int x, y, xSpeed, ySpeed;     // package access
  þÿ // Constructor
þÿ   public MovablePoint(int x, int y, int xSpeed, int ySpeed) {
      this.x = x;
      ......
   }
   // Implement abstract methods declared in the interface Movable
  þÿ@Override
  public void moveUp() {
      y -= ySpeed;   // y-axis pointing down for 2D graphics
   }
   ......
}

class MovableCircle implements Movable
{
   private MovablePoint center;   // can use center.x, center.y directly because they are
package accessible
   private int radius;
   // Constructor
   þÿpublic MovableCircle(int x, int y, int xSpeed, int ySpeed, int radius) {
      // Call the MovablePoint's constructor to allocate the center instance.
      center = new MovablePoint(x, y, xSpeed, ySpeed);
      ......
   }
   ......
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
   // Implement abstract methods declared in the interface Movable
þÿ  @Override
  public void moveUp() {
      center.y -= center.ySpeed;
   }
   .....
}
```

Also add a toString() method in both above classes to print their x and y position in the format shown below: -

[X=value, Y=value] // for MovablePoint

[X=value, Y=value, radius=value] // for MovableCircle

Solution:

```
class MovablePoint implements Movable
{
  int x, y, xSpeed, ySpeed;
  // Constructor
MovablePoint(int x, int y, int xSpeed, int ySpeed){
this.x=x;
this.y=y;
this.xSpeed=xSpeed;
this.ySpeed=ySpeed;
}
 public void moveUp(){
 y-=ySpeed;
 }
  public void moveDown(){
   y+=ySpeed;
  }
  public void moveLeft(){
  x-=xSpeed;
  }
  public void moveRight(){
  x+=xSpeed;
  }
  public String toString(){
  return "[X=" +x+ ", Y=" +y+ "]";
  }
  // Implement abstract methods declared in the interface Movable
}
class MovableCircle implements Movable
{
  private MovablePoint center;
  private int radius;
  // Constructor
MovableCircle(int x, int y, int xSpeed, int ySpeed, int radius){
center=new MovablePoint(x,y,xSpeed,ySpeed);
this.radius=radius;
}
 public void moveUp() {
     center.y -= center.ySpeed;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
  }
   public void moveDown() {
     center.y += center.ySpeed;
   }
   public void moveLeft() {
     center.x -= center.xSpeed;
   }
   public void moveRight() {
     center.x += center.xSpeed;
   }
   public String toString(){
  return "[X="+center.x+", Y="+center.y+", radius="+radius+"]";
  }
  // Implement abstract methods declared in the interface Movable
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Class : Circle and ResizableCircle

You are given
an interface called GeometricObject, which declares two abstract methods: getPerimeter() and getArea() andan interface Resizable declares an abstract method resize(), which modifies the dimension (such as radius) by the given percentage.

```
interface GeometricObject
{
  public double getPerimeter();
  public double getArea();
}
interface Resizable
{
  public void resize(int percent);
}
```

You have to write the implementation of
class Circle, with a protected variable radius, which implements the interface GeometricObject. It includes an parameterized constructor to initialize radius, andclass ResizableCircle as a subclass of the class Circle, which also implements the interface Resizable.
Both your classes must have an method called toString() which returns an string in the format shown below: -
Circle[radius=value]

Sample Input

```
1 // radius of circle
2 // radius of resizable circle
200 // percentage to resize the circle
```

Sample Output

```
Circle[radius=1.0]  // Print the circle
6.28        // Perimeter of circle
3.14        // Area of circle
Circle[radius=2.0]
12.57       // Perimeter of circle
12.57       // Area of circle
25.13       // Perimeter of circle after resize
50.27       // Area of circle after resize
```

Solution:

```
class Circle implements GeometricObject
{
double radius;
Circle(double radius){
this.radius=radius;
}
public double getPerimeter(){
return 2*Math.PI*radius;
}
public double getArea(){
return Math.PI*radius*radius;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
public String toString(){
return "Circle[radius="+radius+"]";
}
}
class ResizableCircle extends Circle implements Resizable
{
ResizableCircle(double radius){
super(radius);
}
 public void resize(int percent){
 radius=radius*percent/100.00;
 }
 public String toString(){
return "Circle[radius="+radius+"]";
}
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Duplicate the Queue elements

Write a function named **doubleQueue()** that accepts
a reference to a queue of integers as a parameter
and replaces every element with two copies of itself.
For example, if a queue named q stores {11, 12, 13}, the call of **doubleQueue**(q); should change it to store {11, 11, 12, 12, 13, 13}.
Constraints: Do not use any auxiliary collections as storage.

Solution:

```
static void doubleQueue(Queue<Integer> q)
{
  int n=q.size();
    for(int i=0;i<n;i++){
        int val=q.remove();
        q.add(val);
        q.add(val);
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Get the mirror of the queue

Write a function named **mirrorQueue()** that accepts
a reference to a queue of strings as a parameter
and appends the queue's contents to itself in reverse order.
For example, if a queue named q stores {"Code", "Quotient"}, the call of **mirrorQueue(q);**
should change it to store {"Code", "Quotient", "Quotient", "Code"}.

Solution:

```
static void mirrorQueue(Queue<String> q)
{
  int n=q.size();
    Stack <String> st= new Stack<>();
    for(int i=0;i<n;i++){
        String val= q.remove();
        st.push(val);
        q.add(val);
    }
    while(!st.isEmpty()){
        q.add(st.pop());
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Check for balanced parentheses in string

Complete the function **balancedString()** that accepts a string of source code and check whether the braces/parentheses are balanced.
Every (or { must be closed by a } or) in the opposite order. Return the index at which an imbalance occurs, or -1 if the string is balanced. If any (or { are never closed, return the string's length.
Here are some example calls:
balancedString("if(a(4)>9){foo(a(2));}") // returns -1 because balanced
balancedString("for(i=0;i<a(3};i++){foo{};)") // returns 13 because } out of order
balancedString("while(true)foo();}{()") // returns 17 because } doesn't match any {
balancedString("if(x){") // returns 6 because { is never closed
Input Format:
Input consists of a single line that contains the string of source code.
Output Format:
Return the index at which the imbalance occurs, if the string is balanced return -1
Sample Input
if(a(4)>9){foo(a(2));}
Sample Output
-1
Explanation
The given string is balanced, hence the output is -1þÿ
Sample Input
for(i=0;i<a(3};i++){foo{};)
Sample Output
13
Explanation
The given string is imbalanced at } on index 13þÿ

Solution:

```
class Result{
  static int balancedString(String s){
    int n=s.length();
      Stack <Character> st= new Stack<>();
      for(int i=0;i<n;i++){
         char ch= s.charAt(i);
         if(ch=='(' || ch=='{'){
            st.push(ch);
         }
         else if(ch==')' || ch=='}'){
            if(st.isEmpty()){
               return i;
            }
            char top=st.pop();
            if((ch=='}' && top!='{') || (ch==')' && top!='(')){
               return i;
            }
         }
      }
      if(!st.isEmpty()){
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            return n;
        }
        return -1;
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Flip the odd elements of queue

Write a function named **flipHalfQueue()** that
Takes an queue of integers as parameter
and reverses the order of half of the elements of a Queue of integers,
Your function should reverse the order of all the elements in odd-numbered positions (position 1, 3, 5, etc.) assuming that the first value in the queue has position 0.
For example, if the queue originally stores this sequence of numbers when the function is called:
index: 0 1 2 3 4 5 6 7
front:{5, 7, 6, 2, 9, 18, 11, 15} back
Then it should store the following values after the function finishes executing:
index: 0 1 2 3 4 5 6 7
front:{5, 15, 6, 18, 9, 2, 11, 7} back
Notice that numbers in even positions (positions 0, 2, 4, 6) have not moved. That sub-sequence of numbers is still: (5, 6, 9, 11).
But notice that the numbers in odd positions (positions 1, 3, 5, 7) are now in reverse order relative to the original. In other words, the original sub-sequence: (7, 2, 18, 15) - has become: (15, 18, 2, 7).
Constraints: You may use a single stack as auxiliary storage.

Solution:

```
static void flipHalfQueue(Queue<Integer> q)
{
  Stack <Integer> st= new Stack<>();
    int n=q.size();
    for(int i=0;i<n;i++){
        int x=q.remove();
        if(i%2==1){
            st.push(x);
        }
        q.add(x);
    }
    for(int i=0; i<n;i++){
        int x=q.remove();
        if(i%2==1){
            q.add(st.pop());
        }
        else{
            q.add(x);
        }
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Number is a Happy number or not

Write a function named **isHappy()** that returns whether a given integer is "happy" or not.
An integer is "happy" if repeatedly summing the squares of its digits eventually leads to the number 1.
For example, 139 is happy because:
12 + 32 + 92 = 9192 + 12 = 8282 + 22 = 6862 + 82 = 10012 + 02 + 02 = 1
By contrast, 4 is not happy because:
42 = 1612 + 62 = 3732 + 72 = 5852 + 82 = 8982 + 92 = 14512 + 42 + 52 = 4242 + 22 = 2022 + 02 = 4...

Note: Don't stuck in an infinite loop.

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name and method signature
class Main
{
  static boolean isHappy(int n)
  {
    // Write your code here
      int slow=n;
      int fast=get(n);
      while(fast!=1 && slow!=fast){
          slow=get(slow);
          fast=get(get(fast));
      }
      return fast==1;
  }
    static int get(int n){
        int sum=0;
        while(n>0){
            int rem=n%10;
            sum+=rem*rem;
            n/=10;
        }
        return sum;
    }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Remove duplicates from a vector

Write a function named removeDuplicates() that accepts as a parameter
a reference to a Vector of integers,
and modifies it by removing any duplicates.
Note that the elements of the vector are not in any particular order, so the duplicates might not occur consecutively.
You should retain the original relative order of the elements.
Use a Set as auxiliary storage to help you solve this problem.
For example, if a vector named v stores {24, 10, 12, 19, 24, 17, 12, 10, 10, 19, 26, 26}, the call of **removeDuplicates(v);** should modify it to store {24, 10, 12, 19, 17, 26}.

Solution:

```
static void removeDuplicates(Vector<Integer> v)
{
HashSet<Integer> set= new LinkedHashSet<>(v);
   v.clear();
   v.addAll(set);
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Map contains same Range or not

Write a function named **boolean hasDuplicateMappings()** that
accepts a Map from strings to strings as a parameter andreturns true if any two keys map to the same value.
For example, if a map named map1 stores {Girdhar=Gopal, Code=Quotient, Hello=Hi, Mere=Gopal}, the call of **hasDuplicateMappings**(map1) would return true because both "Girdhar" and "Mere" map to the value "Gopal".
Return false if passed an empty or one-element map. Do not modify the map passed in.

Solution:

```
public static boolean hasDuplicateMappings(Map<String, String> map)
{
   HashSet <String> set= new HashSet<>();
   for(String val:map.values()){
      if(set.contains(val)){
         return true;
      }
      else{
         set.add(val);
      }
   }
   return false;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Implement Own ArrayList in Java

Write a class to implement your own ArrayList class. It should contain add(), get(), remove(), size() methods.
Use dynamic array logic so that it should increase its size when it reaches threshold.

Solution:

```
import java.util.Scanner;
class OwnArrayList<E>
{
  private static final int INITIAL_CAPACITY = 10;
  private int actSize = 0;
  private Object elementData[];
  public OwnArrayList() {
     elementData = new Object[INITIAL_CAPACITY];
  }
  @SuppressWarnings("unchecked")
  public E get(int index) {
     if (index >= actSize || index < 0) {
        throw new IndexOutOfBoundsException("Index out of bounds");
     }
     return (E) elementData[index];
  }
  public void add(E e) {
     if (actSize == elementData.length) {
        increaseListSize();
     }
     elementData[actSize++] = e;
  }
  @SuppressWarnings("unchecked")
  public Object remove(int index) {
     if (index >= actSize || index < 0) {
        throw new IndexOutOfBoundsException("Index out of bounds");
     }
     Object removedElement = elementData[index];
     for (int i = index; i < actSize - 1; i++) {
        elementData[i] = elementData[i + 1];
     }
     elementData[--actSize] = null;
     return removedElement;
  }
  public int size() {
     return actSize;
  }
  private void increaseListSize() {
     int newCapacity = elementData.length * 2;
     Object newArray[] = new Object[newCapacity];
     for (int i = 0; i < elementData.length; i++) {
        newArray[i] = elementData[i];
     }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
        elementData = newArray;
    }
    public void display() {
        for (int i = 0; i < actSize; i++) {
            System.out.print(elementData[i] + " ");
        }
        System.out.println();
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Implement the stack using array

Implement the stack using array representation

Input

First line of input contains the number of test cases.
First line of each test case contains an integer Q denoting the number of queries.
A Query is of 2 Types
(a) 1 item (a query of this type means push 'item' onto the stack)
(b) 2 (a query of this type means to pop element from stack and print the popped element)
The next Q lines of each test case contains Q queries.

Output

The output for each test case will be space separated integers having -1 if the stack is empty else the element popped out from the stack.
You are required to complete the methods given.

Sample Input

1
8
1
3
2
1
4
1
2
2
1
6
2
2

Sample Output

3 2 6 4

Explanation:

First query is push 3,
then pop will print 3,
3rd query is push 4,
then push 2,
5th query is pop which prints 2,
6th query is push 6,
then pop will print 6 and
last query of pop will print 4.

Solution:

```
class CQStack
{
  public int maxSize; // size of stack array
  public int[] stackArray;
  public int top; // top of stack
  public CQStack(int s) // constructor
  {
     maxSize=s;
     stackArray= new int [s];
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    top=-1;
  }
  public void push(int j) // put item on top of stack
  {
    stackArray[++top]=j;
  }
  public int pop() // take item from top of stack
  {
    if(isEmpty()) return -1;
    return stackArray[top--];
  }
  public boolean isEmpty() // true if stack is empty
  {
    return top==-1;
  }
  public boolean isFull() // true if stack is full
  {
    return top==maxSize;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Reverse a string using stack

Given a string, the task is to reverse the string using stack data structure.
Complete the function **reverseString()** that accepts the string, and reverses the string.

Input Format

The first line of input will contains an integer T denoting the no of test cases. Then T test cases follow.
Each test case contains a number N.
Then N strings follow which are to be reversed

Output Format

For each test case, you have to reverse the string in the array given. You are required to complete the methods given only.

Constraints

1 <= T <= 10
þÿ0 <= N <= 100
Given strings consist of uppercase and lowercase English letters.

Sample Input

1 // No. of test cases
2 // No. of strings
CodeQuotient
Code

Sample Output

tneitouQedoC
edoCþÿ

Solution:

```
/* class CQStack{
     public CQStack(int s) // constructor
     public void push(int j) // put item on top of stack
     public int pop() // take item from top of stack
     public boolean isEmpty() // true if stack is empty
     public boolean isFull() // true if stack is full
   }
   CQStack class already defined as per the above blueprint
*/
static String reverseString(CQStack s, String st)
{
StringBuilder sb= new StringBuilder(st);
   return sb.reverse().toString();
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Implement the stack using Linked List

Implement the stack using linked list representation

Input

First line of input contains number of test cases.
First line of each test case contains an integer Q denoting the number of queries.
A Query is of 2 Types
(a) 1 item (a query of this type means push 'item' onto the stack)
(b) 2 (a query of this type means to pop element from stack and print the popped element)
Then Q lines follow of each test case containing the queries.

Output

The output for each test case will be space separated integers having -1 if the stack is empty else the element popped out from the stack.

Sample Input

1
8
1 3 2 1 4 1 2 2 1 6 2 2

Sample Output

3 2 6 4

Explanation:

First query is push 3,
then pop will print 3,
3rd query is push 4,
then push 2,
5th query is pop which prints 2,
6th query is push 6,
then pop will print 6 and
last query of pop will print 4.

Solution:

```
/* class LinkList
{
  public int data; // data item
  public LinkList next; // next link in list
  public LinkList(int dd) // constructor
  { data = dd; }
} The LinkList class is given as above
*/
class LinkStack
{
  private LinkList first; // ref to first item on list
  public LinkStack() // constructor
  {
     first=null;
  }
  public boolean isEmpty()
  {
     return first==null;
  }
  public void push(int dd)
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
  {
     LinkList res= new LinkList(dd);
     res.next=first;
     first=res;
  }
  public int pop()
  {
     if(first==null) return -1;
     int rem=first.data;
     first=first.next;
     return rem;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Design stack for push, pop and min functions

Implement the stack using array which supports, push, pop and getMin functions with constant time complexity.

Input Format

First Line of Input contains number of test cases
Each test case has Q+1 lines, where first line contains an integer Q denoting the number of queries.
A Query is of 3 Types
(a) 1 item (a query of this type means push 'item' onto the stack)
(b) 2 (a query of this type means to pop element from stack and print the popped element)
(c) 3 (a query of this type means to print the minimum of the stack, it will not remove the element from stack)
The next Q lines of each test case contains Q queries.

Output Format

The output for each test case will be space separated integers having -1 if the stack is empty else the element popped out from the stack.
You are required to complete the methods given.

Sample Input

5
1
8
1
4
1
3
3
2
3
1
1
3
2

Sample Output

3 3 4 1 1

Explanation:

First query is push 4, then 2nd is push 3. So stack is { 3 4 }
3rd query is getmin which prints 3, then pop will print 3. So stack is { 4 }
5th query is getmin which prints 4, then push 1 on stack. So stack is { 1 4 }
7th query is getmin which prints 1, then last query is pop which prints 1.

Solution:

```
/*
int stackArray[maxSize], top = -1,maxSize;
int isFull();
int isEmpty();
Above variables are used for Stack, maxSize and top. Also above two functions are provided.
*/
Stack <Integer> min= new Stack<>();
public void push(int j) // put item on top of stack
{
  if(min.isEmpty() || min.peek()>j){
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
      min.push(j);
  }
    stackArray[++top]=j;
}
public int pop() // take item from top of stack
{
  //if(min.isEmpty()) return -1;
    int rem=stackArray[top--];
    if(rem==min.peek()){
        min.pop();
    }
    return rem;
}
public int getMin()
{
  return min.peek();
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find the next greater element on right side

Given an array having distinct integer elements, the task is to find the first next greater element on right side for each element of the array. If no such element exists, output -1. Complete the function **printNextGreaterElement()** which accepts the array as an argument amd prints the next greater element of each element of array seperated by a space

Input Format
The first line of input contains a single integer T denoting the number of test cases. Then T test cases will follow.
The first line of each test case contains an integer N denoting the size of the array. The next line of each test case contains N positive integers denoting the elements of the array.
Output Format
For each test case, print the next greater element on right side for each element separated by space in new lines.
Sample Input
1
4
1 2 3 4
Sample Output
2 3 4 -1
Explanation
Next greater elements on right side is printed, for last element -1 is printed.

Solution:

```
class Result{
  static void printNextGreaterElement(int arr[],int n){
    // Write your code here
      int res[] = new int [n];
      for(int i=0;i<n;i++){
          int greater=-1;
          for(int j=i+1;j<n;j++){
              if(arr[j] > arr[i]){
                  greater=arr[j];
                  break;
              }
          }
          res[i]=greater;
      }
      for(int p:res){
          System.out.print(p+" ");
      }
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find the minimum bracket reversals for balanced expression

Given an expression having only square brackets '[' and ']'. Find the minimum number of brackets reversals required to make the expression balanced. If expression cannot be made balanced, print -1 and if it is already balanced, print 0.

Input Format

The first line of input contains a single integer T denoting the number of test cases. Then T test cases follow.

Each test case contains an expression consisting of square brackets only.

Output Format

The output for each test case will be the minimum number of bracket reversals required to make the expression balanced if possible.

Sample Input 1

2
[]
][

Sample Output 1

0
2

Explanation 1

First expression is already balanced, so print 0.

Second expression requires 2 reversals (both the brackets needs to be changed as []) so prints 2.

Sample Input 2

1
[[[[

Sample Output 2

2

Sample Input 3

1
[[[[[

Sample Output 3

-1

Solution:

```
class Result
{
  static int minReversal(String expr){
    // Write your code here
      int n=expr.length();
      if(n%2!=0) return -1;
      int open=0, close=0;
      for(int i=0; i<n;i++){
          char ch= expr.charAt(i);
          if(ch=='['){
              open++;
          }
          else{
              if(open>0){
                  open--;
              }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            else{
                close++;
            }
        }
    }
    return (open+1)/2 + (close+1)/2;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Evaluate the postfix expression using Stack

Given a postfix expression, evaluate it using stack and print the final output.

Input Format

The first line of input contains an integer T denoting the number of test cases.
The next T line contains a postfix expression.
An expression in postfix form will consist of all digits and following five operators: +, -, *, /, ^

Output Format

Print the final output of postfix expression evaluation in new line for each test case.

Sample Input

2 // Testcases
8425+-*
546+*493/+*

Sample Output

-24
350

Explanation

Testcase 1 will be evaluated as:
8 * (4 - (2 + 5)) = -24

Testcase 2 will be evaluated as:
(5 * (4 + 6)) * (4 + (9 / 3)) = 350

Solution:

```
/* isEmpty(), isFull(), push(int) and int pop() functions available on Stack. */
static int evalPostfix(CQStack s, String exp) {
  // Write your code here
int n=exp.length();
    for(int i=0;i<n;i++){
        char ch=exp.charAt(i);
        if(Character.isDigit(ch)){
            s.push(ch-'0');
        }
        else{
            int b=s.pop();
            int a=s.pop();
            int res=0;
            switch(ch){
                case '+': res=a+b;
                    break;
                    case '-': res=a-b;
                    break;
                    case '*': res=a*b;
                    break;
                    case '/': res=a/b;
                    break;
                    case '^': res=(int)Math.pow(a,b);
                    break;
            }
            s.push(res);
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            }
        }
        return s.pop();
    }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Evaluate the prefix expression using Stack

Given a prefix expression, evaluate it using stack and print the final output.
Complete the **evalPrefix**() function and return the final output.

Input Format

The first line of input contains an integer T denoting the number of test cases. The next T lines contain a prefix expression.
An expression in prefix form will consist of all digits and following five operators: +, -, *, /, ^

Output Format

Print the final output of prefix expression evaluation in new line for each test case.

Sample Input

1
+-*235/^234

Sample Output

3

Solution:

```
/* class CQStack{
     public CQStack(int s) // constructor
     public void push(int j) // put item on top of stack
     public int pop() // take item from top of stack
     public boolean isEmpty() // true if stack is empty
     public boolean isFull() // true if stack is full
   }
   CQStack class already defined as per the above blueprint
*/
class Result {
  static int evalPrefix(CQStack s, String exp) {
    // Write your code here
      int n=exp.length();
      for(int i=n-1;i>=0;i--){
          char ch=exp.charAt(i);
          if(Character.isDigit(ch)){
              s.push(ch-'0');
          }
          else{
              int a=s.pop();
              int b=s.pop();
              int res=0;
              switch(ch){
                  case '+': res=a+b; break;
                      case '-': res=a-b; break;
                      case '*': res=a*b; break;
                      case '/': res=a/b; break;
                      case '^': res=(int)Math.pow(a,b); break;
              }
              s.push(res);
          }
      }
      return s.pop();
  }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Implement the queue using array

Implement the queue using array representation. Complete the methods given in the editor.

Input Format

First line of input contains an integer T denoting number of test cases. Each test case contains an integer Q denoting the number of queries.

A Query is of 2 Types

(a) 1 <item> (a query of this type means insert 'item' into the queue)

(b) 2 (a query of this type means to delete element from queue and print the deleted element)

The next lines of each test case contains Q queries.

Output Format

The output for each test case will be space separated integers having -1 if the queue is empty else the element deleted from the queue.

Sample Input

```
1
8
1
3
2
1
4
1
2
2
1
6
2
2
```

Sample Output

3 4 2 6

Explanation:

First query is insert 3,
then delete will print 3,
3rd query is insert 4,
then insert 2,
5th query is delete which prints 4,
6th query is insert 6,
then delete will print 2 and
last query of delete will print 6.

Solution:

```
import java.util.*;
class QueueArray
{
  static int SIZE=100;
  static int front=-1;
  static int rear=-1;
  static int array[]=new int[SIZE];
  QueueArray()
  {
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    front=rear=-1;
  }
  // Method to add an item to the queue.
  void enqueue(int item)
  {
      if(front==-1)front=0;
      array[++rear]= item;
  }
  // Method to remove an item from queue.
  int dequeue()
  {
      if(front==-1 || front> rear) return -1;
      return array[front++];
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Reverse a given queue

Given a queue of integer elements, your task is to reverse it.
The function **reverseQueue(queue)** takes the queue as input and reverse it.

Input Format

The first line of input will contains an integer T denoting the no of test cases.
Then T test cases follow.
Each test case contains a number N followed by N number of elements in order in which they will be inserted in queue.

Output Format

For each test case, you have to reverse the queue in the array given. You are required to complete the methods given only.

Constraints

1 <= T <= 10
þÿ1 <= N <= 100

Sample Input

2
4
1 2 3 4
5
10 20 30 50 40

Sample Output

4 3 2 1
40 50 30 20 10

Solution:

```
/*
class QueueArray
{
  static int SIZE=10;
  static int front=-1;
  static int rear=-1;
  static int array[]=new int[SIZE];
  void enqueue(int item);
  void dequeue();
  Abobe is the queue declaration.
*/
static void reverseQueue(QueueArray q){
  // Write your code here
    int n=q.rear-q.front+1;
    Stack<Integer> st= new Stack <>();
    for(int i=0;i<n;i++){
        st.push(q.dequeue());
    }
    while(!st.isEmpty()){
        q.enqueue(st.pop());
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Reverse first N elements of a given queue

Given a queue of integer elements and an integer k, your task is to reverse first k elements of the queue, leaving the other elements in same order.

Input Format

The first line of input will contains an integer T denoting the no of test cases. Then T test cases follow. Each test case contains a number N followed by another number K.
In second line of each test case N number of elements are given in order in which they will be inserted in queue.
The function void reverseKelementsQueue(int q[], int K) takes the queue and K as input and reverse first K elements of the queue.

Output Format

For each test case, you have to reverse the first k elements of queue in the array given. You are required to complete the methods given only.

Sample Input

```
2
4 2
1 2 3 4
5 3
10 20 30 40 50
```

Sample Output

```
2 1 3 4
30 20 10 40 50
```

Solution:

```
/*
class QueueArray
{
  int SIZE=50;
  int front=-1;
  int rear=-1;
  int array[]=new int[SIZE];
  void enqueue(int item);
  void dequeue();
  Abobe is the queue declaration.
*/
static void reverseKelementsQueue(QueueArray q, int K)
{
int n=q.rear- q.front+1;
    Stack <Integer> st= new Stack<>();
    for(int i=0;i<K;i++){
        st.push(q.dequeue());
    }
    while(!st.isEmpty()){
        q.enqueue(st.pop());
    }
    int rem=n-K;
    for(int i=0;i<rem;i++){
        q.enqueue(q.dequeue());
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Print the List

Given a pointer to the head node of a linked list, print its elements in forward and backward order, separated by hyphen. If the head pointer is null (indicating the list is empty), don't print anything.
The functions **void forwardPrint()** & **void backwardPrint()** takes the head node of a linked list as a parameter.
Note: Do not read any input from stdin/console. Each test case calls the forwardPrint & backwardPrint method individually and passes it the head of a list.
Input Format
First line contains the number of test cases.
Each test case contains an integer denoting the number of elements in list, followed by the numbers in new lines.
Output Format
Print the integer data for each element of the linked list separated by hyphen.
Constraints
1 <= T <= 10
þÿ1 <= N <= 10^8
þÿ-(10^9) <= List_Element <= 10^9
Sample Input
1 // Test Cases
3 // N
1 // List Elements
2
3
Sample Output
1-2-3-
3-2-1-

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node next;
 *  };
 *
 *  The above class defines the linked list node.
 */
static void forwardPrint(Node head) {
if(head==null) return;
   while(head!=null){
       System.out.print(head.data+"-");
       head=head.next;
   }
}
static void backwardPrint(Node head) {
if(head==null) return;
   backwardPrint(head.next);
     System.out.print(head.data+"-");
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Copy first list to second list

Given a pointer to the head node of a linked list, copy all the elements in another list and return the head of second list.

Input Format:

First line contains one integer denoting the number of test cases.
For each test case, first line contains the total number of nodes in list i.e. N and next N lines contains the elements of nodes.
The function copyList() takes the head node of a linked list as a parameter and return the head pointer of copied list.
Note: Do not read any input from stdin/console. Each test case calls the copyList method individually and passes it the head of a list.

Output Format:

Print the integer data for each element of the linked list separated by space.

Sample Input

```
1  // Test Cases
3  // No. of nodes
1  // List Elements
2
3
```

Sample Output

```
1 2 3
```

Solution:

```
/*
  class Node {
    int data;
    Node next;
    Node(){}
    Node(int d) {
      data=d;
    }
  }
  The above class defines the linked list node.
*/
static Node copyList(Node head) {
  // Write your code here
  if(head==null) return null;
    Node newhead= new Node(head.data);
    Node curr= head.next;
    Node temp=newhead;
    while(curr!=null){
        temp.next= new Node(curr.data);
        curr=curr.next;
        temp=temp.next;
    }
    return newhead;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Move the Smallest and largest to head and tail of list

Given a pointer to the head node of a linked list, find the smallest and the largest element of this list. Now, move the smallest node to the front and move the largest node to the end of the list.

Example:

Input list: head -> 5 -> 4 -> 7 -> 3 -> 2 -> 6 -> NULL
Output list: head -> 2 -> 5 -> 4 -> 3 -> 6 -> 7 -> NULL
Complete the function **shiftSmallLarge()**, which takes the head node of the linked list as a parameter, and returns the updated head pointer after doing both the shifts.

Input Format:

First line contains one integer denoting the number of test cases.
For each test case, first line contains the total number of nodes in list i.e. N and next N lines contains the elements of nodes.

Output Format:

Print the integer data for each element of the linked list separated by space.

Constraints:

1 <= no. of testcases <= 10
0 <= no. of nodes <= 100
0 <= node data <= 1000

Sample Input

1 // Test Cases
7 // N (testcase 1)
12 // linked list elements
8
6
20
1
50
16

Sample Output

1 12 8 6 20 16 50

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node next;
 *  };
 *
 *  The above class defines the linked list node.
 */
// Return the head of updated list
static Node shiftSmallLarge(Node head) {
  // Write your code here
if(head==null) return null;
    Node min=head, prevmin=null;
    Node curr=head, prev=null;
    while(curr!=null){
        if(curr.data<min.data){
            min=curr;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            prevmin=prev;
        }
        prev=curr;
        curr=curr.next;
    }
    if(min!=head){
        if(prevmin!=null){
         prevmin.next=min.next;
        }
        min.next=head;
        head=min;
    }
    Node max=head, prevmax=null;
    curr=head;
    prev=null;
    while(curr!=null){
        if(curr.data>max.data){
            max=curr;
            prevmax=prev;
        }
        prev=curr;
        curr=curr.next;
    }
    if(max.next!=null){
        if(prevmax!=null){
            prevmax.next=max.next;
        }
        prev.next=max;
        max.next=null;
    }
    return head;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Check List for Palindrome

Given a pointer to the head node of a linked list, check it is a palindrome or not.
Examples:
Palindrome List: head -> 702 -> 59 -> 702 -> NULL
Palindrome List: head -> 3 -> 4 -> 4 -> 3 -> NULL
Non Palindrome List: head -> 1 -> 2 -> 3 -> 1 -> NULL
Complete the function **checkPalindrome()** which takes the head node of a linked list as a parameter, and return 1 if the given list is a palindrome, and 0 otherwise.
Note: If the linked list is empty, return 0.
Input Format:
The first line will contains an integer i.e. number of test cases
Each test case contains two line. First line denotes the number of nodes in list and second line contains the node values.
Output Format:
Print 1 if list is palindrome and 0 if list is not palindrome.
Constraints:
1 <= no. of testcases <= 10
0 <= no. of nodes <= 10^5
0 <= node data <= 10^6
þÿSample Input
2 // Test Cases
3
1 2 3
3
1 2 þÿ1
Sample Output
0
1

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node next;
 *  };
 *
 *  The above class defines the linked list node.
 */
class Result {
  static int checkPalindrome(Node head) {
    // Write your code here
    if(head==null) return 0;
      ArrayList <Integer> list= new ArrayList<>();
      Node temp=head;
      while(temp!=null){
          list.add(temp.data);
          temp=temp.next;
      }
      int i=0;
      int j=list.size()-1;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
        while(i<j){
            if(!list.get(i).equals(list.get(j))){
                return 0;
            }
            i++;
            j--;
        }
        return 1;
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find the loop in Linked list

A linked list is a collection of nodes stored in memory through pointers. These pointers may or may not be replicated, which might result in a loop of nodes.
Given a linked-list as input, check whether it contains a loop or not. If there is a loop return the number of nodes in the loop, otherwise return 0.
Complete the function **loopInList()** which takes the head node of a linked list as a parameter, and returns the number of nodes in loop if exists, 0 otherwise.

Input Format

First line contains the number of Test cases i.e. T.
Each test case contains the following:
First line will contain N, denoting the number of nodes.
Next N lines will contain the node's data.
Last line will contain the starting position of loop in the list. If no loop exists, then this input will be -1.

Output Format

Print the number of nodes in the list involved in a loop.

Constraints

1 <= no. of testcases <= 10
0 <= no. of nodes <= 10^5
þÿ0 <= node data <= 1000

Sample Input

```
1  // test cases
8    // no. of nodes (TC-1)
5 4 9 3 10 2 6 7
2    // This signifies the starting position of the loop (consider 0-based indexing)
```

Sample Output

6

Explanation

As 9 -> 3 -> 10 -> 2 -> 6 -> 7 -> 9 .... is the loop and 6 nodes are involved.

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node next;
 *  };
 *
 *  The above class defines the linked list node.
 */
class Result {
  static int loopInList(Node head) {
    // Write your code here
    if(head==null) return 0;
      Node slow=head;
      Node fast= head;
      while(fast!=null && fast.next!=null){
          slow=slow.next;
          fast=fast.next.next;
          if(slow==fast){
              Node temp=slow.next;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            int n=1;
            while(temp!=slow){
               n++;
               temp=temp.next;
            }
            return n;
         }
      }
      return 0;
   }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Reverse a linked list

A linked list is a collection of nodes. First node is called head node and last node called the tail node.
Now, given a pointer to the head node of a linked list, can you reverse the linked list i.e. New list must start with tail node and end at head node. So it becomes the reverse of the original list.

Example:
Original List: head -> 1 -> 2 -> 3 -> 4 -> 5 -> NULL
Reversed List: head -> 5 -> 4 -> 3 -> 2 -> 1 -> NULL
Complete the function **reverseList()** given in the editor, which takes the head node of a linked list as a parameter, and returns the new head of reversed list.

Input Format:
First line contains an integer i.e. Number of test cases.
Each test case have two lines. First line has the total number of nodes and second line has the node values.

Output Format:
Print the integer data for each element of the reversed linked list separated by space.

Constraints:
1 <= no. of testcases <= 10
0 <= no. of nodes <= 1000
0 <= node data <= 100þÿþÿ

Sample Input
1 // Test Cases
3 // no. of nodes
1 2 3 // node's data

Sample Output
3 2 1

Solution:

```
/*
  class Node {
    int data;
    Node next;
    Node(){}
    Node(int d) {
      data=d;
    }
  }
  The above class defines the linked list node.
*/
class Result {
  // Return the new head of reversed list
  static Node reverseList(Node head) {
    // Write your code here
    if(head==null )return null;
      Node prev=null, curr=head, next;
      while(curr!=null){
          next=curr.next;
          curr.next=prev;
          prev=curr;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            curr=next;
        }
        return prev;
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Add Two Numbers Represented by Lists

An n-digit number can be expressed using a linked list of n nodes, where each node can be used to represent a digit.
Now given two numbers represented by two singly linked lists, compute the sum of these numbers, provided that the digits are stored in reverse order, such that the digit of 1's place is at the head of the list.

Example 1:
number 254 will be represented as: head -> 4 -> 5 -> 2 -> NULL
number 71 will be represented as: head -> 1 -> 7 -> NULL
Their sum 325 is represented as: head -> 5 -> 2 -> 3 -> NULL

Example 2:
number 86 will be represented as: head -> 6 -> 8 -> NULL
number 37 will be represented as: head -> 7 -> 3 -> NULL
Their sum 123 is represented as: head -> 3 -> 2 -> 1 -> NULL

Complete the function **addListNumbers**() which takes the head nodes of two linked lists as parameters, and return the head of the list which contains the computed sum.

Input Format:
First line contains an integer denoting the number of test cases.
Each test cases has 4 lines:
First line contains the digit count of first number and second line contains the digits.
Third line contains the digit count of second number and fourth line contains the digits.

Output Format:
Print the integer data for each element of the sum list separated by space.

Constraints:
1 <= no. of test cases <= 10
0 <= no. of nodes <= 1000
0 <= node data <= 9þÿþÿ

Sample Input
```
2  // Test Cases
3    // testcase 1
4 1 5
3
5 9 2
3    // testcase 2
3 1 3
2
2 1
```

Sample Output
```
9 0 8
5 2 3
```

Explanation:
For first two numbers, they are 514 and 295 so the sum will be 809
For second two numbers, they are 313 and 12 so the sum will be 325

Solution:

```
/*
  class Node {
    int data;
    Node next;
    Node(){}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    Node(int d) {
      data=d;
    }
  }
  The above class defines the linked list node.
*/
class Result {
  // Return the head of sum list
  static Node addListNumbers(Node head1, Node head2) {
    // Write your code here
   if(head1==null) return head2;
      if(head2==null) return head1;
      Node ans= new Node(0);
      Node curr=ans;
      int carry=0;
      while(head1!=null || head2!=null || carry!=0){
          int sum=carry;
          if(head1!=null){
              sum+=head1.data;
              head1=head1.next;
          }
          if(head2!=null){
              sum+=head2.data;
              head2=head2.next;
          }
          carry=sum/10;
          curr.next= new Node(sum%10);
          curr=curr.next;
      }
      return ans.next;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Delete a Node in Linked list given access to only that Node

A linked list is a collection of nodes. Where each node contains some data address of the next node, if it is the last node then it contains the data only. This way we can access all the nodes if we have the address of first node.
Now the task is that given a pointer to some node in a linked list, delete it if it is not the last node of the list.
Complete the function **deleteNode()** which takes the address of the node of a linked list as a parameter and delete this node from the list. (If given node is the last node of the list or if the list is empty, then do nothing.)

Input Format

First line will contain the number of test cases T.
Each test case contains 3 lines. First line has the total number of nodes and second line contains the N values for the nodes.
Third line contains the node number after the head node to be deleted.

Output Format

Print the integer data for each element of the modified linked list separated by space.

Constraints

1 <= no. of testcases <= 10
0 <= no. of nodes <= 10^5
þÿþÿ0 <= node data <= 10^6

Sample Input

1 // Test Cases
4
1 5 6 8
2

Sample Output

1 5 8

Explanation:

2 means 2nd node after head is to be deleted, so node with value 6 is deleted.

Solution:

```
/*
  class Node {
    int data;
    Node next;
    Node(){}
    Node(int d) {
      data=d;
    }
  }
  The above class defines the linked list node.
*/
class Result {
  static void deleteNode(Node n1) {
    // Write your code here
if(n1==null || n1.next==null) return;
      n1.data=n1.next.data;
      n1.next=n1.next.next;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Swap Two Nodes of Doubly Linked List

Given a pointer to the head node of a doubly linked list and two keys, **x** and **y** respectively, swap these two nodes of the list (if these nodes exists in the list).
Note: All the linked list nodes contain distinct data.
Complete the function **swapNodes(),** which takes the head node of a doubly linked list along with x and y as a parameter, and returns the head of updated list after swapping the two nodes.
Input Format:
First line contains an integer denoting number of test cases.
Each test case has 4 lines. First line contains the number of elements in list, Second line contains the list elements separated by space.
Third and Fourth lines contains the node numbers to be swapped.
Output Format:
Print the integer data for each element of the modified linked list separated by space.
Constraints:
1 <= no. of testcases <= 10
0 <= no. of nodes <= 10^5
þÿþÿ0 <= node data <= 10^6
0 <= x, y <= 10^6þÿ
Sample Input

```
1           // test cases
6           // no. of nodes (TC-1)
1 2 3 4 5 6   // node's data
3           // x
5           // y
```

Sample Output

```
1 2 5 4 3 6
```

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node next;
 *    Node prev;
 *  };
 *
 *  The above class defines the linked list node.
 */
// Return the head of updated list after swapping the two nodes
Node swapNodes(Node head, int x, int y) {
  // Write your code here
    if(head==null) return null;
  if(x==y) return head;
    Node a=head;
    Node b=head;
    while(a!=null && a.data!=x){
        a=a.next;
    }
    while(b!=null && b.data!=y){
        b=b.next;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    }
    if(a!=null && b!=null){
        int temp=a.data;
        a.data=b.data;
        b.data=temp;
    }
    return head;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Rotate the Doubly Linked List by K elements

Given a pointer to the head node of a doubly linked list and an integer K, rotate the list to the right by k position.

Input Format

The function rotateByK() takes the head node of a doubly linked list and an integer k as parameters, rotate the list to the right by k position and return the head of modified list.

Note: Do not read any input from stdin/console. Each test case calls the rotateByK method individually and passes it the head of a list.

Output Format

Print the integer data for each element of the rotated doubly linked list separated by space.

Constraints

0 <= N <= 10^5

þÿ0 <= K <= N

Sample Input

1 // No. of test cases

7 // No. of elementsþÿ

1

2

3

4

5

6

þÿ7

4 // Value of k

Sample Output

4 5 6 7 1 2 3 // List after rotating to the right by k position

Solution:

```
/* class LinkList
{
  int data;
  LinkList next;
  LinkList prev;
} */
static LinkList rotateByK(LinkList head, int k)
{
if(head==null || k==0) return head;
    LinkList tail=head;
    int n=1;
    while(tail.next!=null){
        n++;
        tail=tail.next;
    }
    tail.next=head;
    head.prev=tail;
    k=k%n;
    LinkList newhead= head;
     int steps=n-k;
    while(steps-->0){
        newhead=newhead.next;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    }
    LinkList newtail=newhead.prev;
    newtail.next=null;
    newhead.prev=null;
    return newhead;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Rearrange the Even-Odd Nodes of Doubly Linked List

Given a pointer to the head node of a doubly linked list, rearrange the nodes of list such that all even positioned nodes are shifted before odd positioned nodes while keeping their relative order same as in the original list.
Note: The head node of the list is at position 1 i.e. odd position
Complete the function **rearrangeList()**, which takes the head node of a doubly linked list as a parameter, and returns the head of updated list after rearranging it.
Input Format:
First line will contain the number of test cases i.e. T.
Each test case consists of two lines. In first line total number of nodes is given and in second line the node values are provided.
Note: Do not read any input from stdin/console. Each test case calls the **rearrangeList()** method individually and passes it the head of a list.
Output Format:
Print the integer data for each element of the rearranged linked list separated by space.
Constraints:
1 <= no. of testcases <= 10
0 <= no. of nodes <= 10^5
þÿþÿþÿ0 <= node data <= 10^6
Sample Input
1 // testcases
7 // no. of nodes (TC-1)
1 3 5 7 9 11 13 // node's data
Sample Output
3 7 11 1 5 9 13
Explanation:
In the given list: head -> 1 -> 3 -> 5 -> 7 -> 9 -> 11 -> 13
3, 7 and 11 are at positions 2, 4 and 6
1, 5, 9 and 13 are at positions 1, 3, 5 and 7

Solution:

```
/*
  *  class Node {
  *    int data;
  *    Node next;
  *    Node prev;
  *  };
  *
  *  The above class defines the linked list node.
  */
 static Node rearrangeList(Node head) {
  // Write your code here
  if(head==null || head.next==null) return head;
    Node oddhead=head, oddtail=head;
    Node evenhead=head.next, eventail=head.next;
    Node curr=head.next.next;
    boolean odd=true;
    while(curr!=null){
       if(odd){
          oddtail.next=curr;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            oddtail=curr;
        }
        else{
            eventail.next=curr;
            eventail=curr;
        }
        odd=!odd;
        curr=curr.next;
    }
    eventail.next=oddhead;
    oddtail.next=null;
    return evenhead;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Given list is circular or not

In circular list the last node points to the first node creating a loop of nodes. Now, given a pointer to the head node of a linked list, find out if it is circular or not.
Complete the function **isCircular()** given in the editor**,** which takes the head of the list as parameter and returns **1** if the linked list is circular and **0** otherwise.
Note: If the linked list is empty, then consider it as a circular linked list.
Input
The functions isCircular() takes the head node of a linked list as input.
Output
Return 1 if the given list is circular, else return 0.
Constraints
0 <= no. of nodes <= 10^5
0 <= node data <= 10^6

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node next;
 *  };
 *
 *  The above class defines the linked list node.
 */
static int isCircular(Node head) {
  // Write your code here
  if(head==null) return 1;
    if(_last.next==head) return 1;
    else return 0;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Insert Nodes in a Circular Linked List

Given a pointer to the head node of a circular linked list, insert the new nodes at beginning or end.
Complete The functions **insertBeg()** & **insertEnd()** takes the head node of a linked list and the data to be inserted as parameters insert the specified node and return the new head.
Note: Do not read any input from stdin/console. Each test case calls the insertBeg & insertEnd method individually and passes it the head of a list.
Input Format:
The first line of input consists of an integer T denoting the number of test cases
Then T test cases follow.
First Line of each test contains number of Queries Q
A Query is of 2 types
(a) 1 data: Inserts the data at the beginning of the circular linked list.
(b) 2 data: Inserts the data at the end of the circular linked list.
Output Format:
Print the integer data for each element of the linked list separated by space.
Sample Input
2 // No of Testcases
5 // No of Queries in test case-1
1 // Query-1
1
1 // Query-2
2
1 // Query-3
3
2 // Query -4
4
2 // Query -5
5
3 // No of Queries in test case-2
2 // Query -1
4
1 // Query -2
2
2 // Query -3
6
Sample Output
3 2 1 4 5
2 4 6

Solution:

```
/* class LinkList
   {
     int data;
     LinkList next;
     LinkList(int d)
     {
       data=d;
     }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
} */
static LinkList insertBeg(LinkList head, int data) {
    // Write your code here
    LinkList newNode= new LinkList (data);
     if(head==null){
         newNode.next=newNode;
         return newNode;
     }
    LinkList tail=head;
    while(tail.next!=head){
       tail=tail.next;
    }
    tail.next=newNode;
    newNode.next=head;
    return newNode;
}
static LinkList insertEnd(LinkList head, int data) {
  // Write your code here
     LinkList newNode= new LinkList (data);
     if(head==null){
         newNode.next=newNode;
         return newNode;
     }
    LinkList tail=head;
    while(tail.next!=head){
       tail=tail.next;
    }
    tail.next=newNode;
    newNode.next=head;
    return head;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Delete in Circular Linked List

Given a pointer to the head node of a circular linked list, delete the nodes from beginning or end.
Complete the functions **deleteBeg()** & **deleteEnd()** that takes the head node of a linked list as parameters delete the specified node and return the new head.
Note: Do not read any input from stdin/console. Each test case calls the deleteBeg & deleteEnd method individually and passes it the head of a list.
Input Format:
First Line contains the number of Nodes in the linked list,N.
Then in the next line, N spaced integers are provided , denoting the elements of the linked list, inserted in beginning of the linked list.
In the next line , an integer , Q is given, denoting the number of queries.
Then Q lines follow, denoting the Query

Query is of 2 types:
(a) 1 : deletes the starting element from the linked list
(b) 2 : deletes the ending element from the linked list
þÿOutput Format:
Print the integer data for each element of the linked list separated by space after performing all the queries.
Sample Input
10 // Number of Nodes in linked list
10 9 8 7 6 5 4 3 2 1 // Nodes in linked list
4 // Number of Queries
1 // Query - 1
2 // Query -2
1 // Query -3
2 // Query -4
þÿSample Output
3 4 5 6 7 8
Explanation:
Initially the list is 1->2->3->4->5->6->7->8->9->10
After Query-1 list is 2->3->4->5->6->7->8->9->10
After Query-2 list is 2->3->4->5->6->7->8->9
After Query-3 list is 3->4->5->6->7->8->9
After Query-4 list is 3->4->5->6->7->8
The output is 3->4->5->6->7->8

Solution:

```
/* class LinkList
{
  int data;
  LinkList next;
} */
static LinkList deleteBeg(LinkList head){
  // Write your code here
    if(head==null || head.next==null) return null;
    LinkList tail=head;
    while(tail.next!=head){
        tail=tail.next;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    }
    tail.next=head.next;
    head=head.next;
    return head;
}
static LinkList deleteEnd(LinkList head){
  // Write your code here
     if(head==null || head.next==null) return null;
     LinkList tail=head;
    while(tail.next.next!=head){
        tail=tail.next;
    }
    tail.next=head;
    return head;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Count the Number of Nodes in Circular Linked List

In circular list the last node points to the first node creating a loop of nodes. Now, given a pointer to the head node of a circular linked list, count the total number of elements in it. Complete the function **countNodes()** which takes the head node of a circular linked list as a parameter and return the number of nodes in the list.

Input Format:

First line will contain the total number of test cases.
Each test case consists of 2 lines, First line has the number of nodes and second line contains the node values separated by spaces.

Constraints:

1 <= Test Cases <= 100
0 <= no. of nodes <= 10^5
0 <= node dataþÿ <= 10^9

Output Format:

Print the total number of nodes in circular list.

Sample Input

```
1  // Test Cases
3      // no. of nodes (TC-1)
1 2 3  // node's data
```

Sample Output

```
3
```

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node next;
 *  };
 *
 *  The above class defines the linked list node.
 */
class Result {
  static int countNodes(Node head) {
    // Write your code here
    if(head==null) return 0;
      int count=1;
     Node tail=head;
      while(tail.next!=head){
          count++;
          tail=tail.next;
      }
      return count;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Insert in a sorted circular linked list

A list is called sorted if all successive nodes have non-decreasing/non-increasing values. Now given a pointer to the head node of a sorted circular linked list, insert an element in it. Complete the function **insertSorted()** which takes the head node of a circular linked list and the data to be inserted and insert the data in a new node at proper position preserving the sorted property and return the new head.

Input Format:

First line contains an integer denoting the number of test cases.
Each test case has 3 lines. First line has the number of nodes in sorted list, second line contains the elements of list in non-increasing order.
Third line contains a new integer to be inserted in the list.

Output Format:

Print the nodes of circular lists separated by space.

Sample Input

1
5
7 5 4 2 1
6

Sample Output

1 2 4 5 6 7

Solution:

```
/* class LinkList {
  int data;
  LinkList next;
  LinkList() {}
  LinkList(int d) {
    data=d;
  }
} */
class Result {
  static LinkList insertSorted(LinkList head, int data) {
LinkList node= new LinkList(data);
      if(head==null){
          node.next=node;
          return node;
      }
      LinkList tail=head;
      while(tail.next!=head){
          tail=tail.next;
      }
      LinkList curr=head;
      if(curr.data>data){
          node.next=head;
          tail.next=node;
          return node;
      }
      while(curr.next!=head && curr.next.data<data){
          curr=curr.next;
      }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
        node.next=curr.next;
        curr.next=node;
        return head;
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Split the Circular Linked List in two parts

A circular linked list is a list in which the last node points back to the first node. It creates a kind of loop in the list hence it is called circular list.
Now, given a pointer to the head node of a circular linked list with even number of nodes, cut it from middle and return the two lists as two separate circular linked lists.
Complete the function **listCut()** which takes the head node of a circular linked list, so you need to cut the list in two parts and return the pointer to second half of list. The first part will remain in the pointer head.

Input Format:

First line will contain an integer denoting number of test cases.
Each test case will have 2 lines, In first line total number of nodes is written and in 2nd line the node values are given.

Output Format:

Print the nodes of two circular lists separated by space on two lines.

Sample Input

1
4
1 2 3 4

Sample Output

1 2
3 4

Solution:

```
/* class LinkList {
  int data;
  LinkList next;
  LinkList() {}
  LinkList(int d) {
    data=d;
  }
} */
class Result {
  static LinkList listCut(LinkList head) {
    // Write your code here
      if(head==null || head.next==null) return null;
      LinkList slow=head;
      LinkList fast=head;
      while(fast.next!=head && fast.next.next!=head){
          slow=slow.next;
          fast=fast.next.next;
      }
      LinkList second=slow.next;
      slow.next=head;
      LinkList tail=second;
      while(tail.next!=head){
          tail=tail.next;
      }
      tail.next=second;
      return second;
  }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Create a binary tree from array

Given an array of integer elements, create a binary tree from this array in level order fashion.
Input Format
The array of elements is given and you have to build the tree from it in level order i.e. elements from left in the array will be filled in the tree level wise starting from level 0.
Do not write the whole program, just complete the function buildTree(int arr[], int n) which takes the array and total number of nodes as parameter and return the root of the binary tree.

Note: Do not read any input from stdin/console. Just complete the functions provided. You can write more functions if required, but just above the given function.
þÿOutput Format
Print the tree in inorder.
Constraints
0 <= n <= 10^5
Sample Input
6 // n
1
2
3
4
5
6
Sample Tree

```
      1
     / \
    2   3
   / \ /
  4 5 6
```

Sample Output
4 2 5 1 6 3

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
// Return the root node of the tree
static Node buildTree(int arr[], int n) {
  Node root = null;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
  // Complete the function body
    if(n==0)return null;
root= new Node(arr[0]);
    Queue <Node> q= new LinkedList<>();
    q.add(root);
    for(int i=1;i<n;){
        Node curr= q.remove();
        curr.left= new Node(arr[i++]);
        q.add(curr.left);
        if(i<n){
            curr.right= new Node(arr[i++]);
            q.add(curr.right);
        }
    }
  return root;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Print binary tree with level order traversal

Given a binary tree, print all nodes of the tree in level order traversal. print nodes at same level separated by space and give new line between every level. **(There should be no space after last node of each level)**

Input

The root node of binary tree is given to you, and you have to complete the function void printLevelWise(Node root) to print the nodes at all levels of the binary tree.

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

Output

Print the nodes level wise separated by space and new line. There should be no space after last node of each level.

Constraints

0 <= no. of nodes <= 10^5

Sample Input

```
      1
     / \
    2   3
   / \ /
  4 5 6
```

Sample Output

```
1
2 3
4 5 6
```

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
static void printLevelWise(Node root) {
  // Write your code here
  if(root==null) return;
    Queue <Node> q= new LinkedList<>();
    q.add(root);
    while(!q.isEmpty()){
        int n= q. size();
        for(int i=0;i<n;i++){
            Node curr=q.remove();
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            System.out.print(curr.data);
            if(i<n-1) System.out.print(" ");
            if(curr.left!=null) q.add(curr.left);
            if(curr.right!=null) q.add(curr.right);
        }
        System.out.println();
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Print nodes at odd levels of the binary tree

Given a binary tree, print all nodes at odd levels of the tree. Assume the root node is at level 1 i.e. odd level.
Complete the function **printOdd()** which will take the root node of the tree as parameter and print the nodes at odd levels of the binary tree.

Input Format

First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.

Output Format

Print the nodes at odd levels separated by space.

Constraints

0 <= N <= 10^5

Sample Input

6
1 2 3 4 5 6

Sample Output

1 4 5 6

Explanation:

```
      1          // level-1
     / \
    2   3
   / \ /
  4  5 6         // level-3
```

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
  static void printOdd(Node root) {
    // Write your code here
    if(root==null) return;
      Queue <Node> q= new LinkedList<>();
      q.add(root);
      int level=1;
      while(!q.isEmpty()){
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            int n=q.size();
            for(int i=0;i<n;i++){
                Node curr= q.remove();
                if(level%2!=0) System.out.print(curr.data+" ");
                if(curr.left!=null) q.add(curr.left);
                if(curr.right!=null) q.add(curr.right);
            }
            level++;
        }
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Write iterative version of inorder traversal

Given a binary tree, print it in inorder fashion.

Input Format

The root node of binary tree is given to you, and you have to complete the function void printInorder(Node root) to print the nodes of tree. (The function should use iteration not recursion).

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

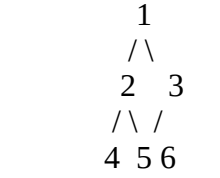
Output Format

Print the nodes of tree separated by space.

Constraints

0 <= N <= 10^5

Sample Input



Sample Output

4 2 5 1 6 3

Solution:

```
/* class Node {
  int data; // data used as key value
  Node leftChild;
  Node rightChild;
  public Node()  {
    data=0;  }
  public Node(int d)  {
    data=d;  }
 } Above class is declared for use. */
static void printInorder(Node root)
{
if(root==null) return;
    Stack <Node> st= new Stack<>();
    Node curr= root;
    while(curr!=null || !st.isEmpty()){
        while(curr!=null){
            st.push(curr);
            curr=curr.leftChild;
        }
        curr=st.pop();
        System.out.print(curr.data+" ");
        curr=curr.rightChild;
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Complete the inorder(), preorder() and postorder() functions for traversal with recursion

Given a binary tree, print it in inorder, preorder and postorder fashion with recursion.

Input

The root node of binary tree is given to you, and you have to complete the function void inorder(Node root), void preorder(Node root) & void postorder(Node root) to print the nodes of tree. (The function should use recursion).

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

Output

Print the nodes of tree separated by space by all three traversals in new lines.

Sample Input

6
1 2 3 4 5 6

Above input corresponds to below tree.

```
      1
     / \
    2   3
   / \ /
  4  5 6
```

Sample Output

4 2 5 1 6 3
1 2 4 5 3 6
4 5 2 6 3 1

Solution:

```
/* class Node {
  int data; // data used as key value
  Node leftChild;
  Node rightChild;
  public Node()  {
    data=0;  }
  public Node(int d)  {
    data=d;  }
 } Above class is declared for use. */
static void inorder(Node root)
{
if(root==null) return;
    inorder(root.leftChild);
    System.out.print(root.data+" ");
    inorder(root.rightChild);
}
static void preorder(Node root)
{
if(root==null) return;
    System.out.print(root.data+" ");
    preorder(root.leftChild);
    preorder(root.rightChild);
}
static void postorder(Node root)
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
{
if(root==null) return;
    postorder(root.leftChild);
    postorder(root.rightChild);
    System.out.print(root.data+" ");
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Construct tree from given inorder and post order traversal

Given two array representing the inorder and postorder traversal of a binary tree, construct the tree and return its root node.

Input

Two arrays for traversals and total number of nodes are given and you needs to write a function Node buildTree(int in[], int post[], int N), which accepts the inorder and postorder traversals of a binary tree and the number of nodes in the tree. The function returns the root node of the constructed tree.

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

Output

The nodes of tree will be printed separated by space using preorder traversal in new lines.

Sample Input

6 // Number of nodes
4 // inorder traversal
2
5
1
6
þÿ3
4 // Postorder traversal
5
2
6
3
1

Sample Output

1 2 4 5 3 6

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node leftChild;
 *    Node rightChild;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
    static int postindex;
  static Node buildTree(int in[], int post[], int N) {
    // Write your code here
    postindex=N-1;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
        return createTree(in, post, 0, N-1);
    }
     static Node createTree(int inorder[], int postorder[], int start, int end){
        if(start>end) return null;
        Node root= new Node(postorder[postindex--]);
        int pos=start;
        while(inorder[pos]!=root.data){
            pos++;
        }
        root.rightChild=createTree(inorder, postorder,pos+1, end );
        root.leftChild= createTree(inorder, postorder, start, pos-1);
        return root;
     }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Count the number of leaf and non-leaf nodes in a binary tree

A leaf in a tree is a node which has no children, i.e. for a binary tree, both its left and right point to NULL. Given a binary tree count the number of leaf and non-leaf nodes in it. Complete the functions **countLeafs()** & **countNonLeafs()** which takes the address of the root node as parameter and return the count of the leaves and non-leafs respectively.

þÿInput Format

First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.

Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.

Output Format

For each test case, print the number of leaf and non-leaf nodes separated by space in new lines.

Constraints

0 <= N <= 10^5

Sample Input

7

1 2 3 4 5 -1 6

Sample Output

3 3

Explanation:

The above tree is:

```
      1
    /  \
   2    3
  / \    \
 4   5    6
```

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
  static int countLeafs(Node root) {
if(root==null) return 0;
    if(root.left==null && root.right==null) return 1;
      return countLeafs(root.left) + countLeafs(root.right);
  }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
  static int countNonLeafs(Node root) {
if(root==null) return 0;
      if(root.left==null && root.right==null) return 0;
      return 1+countNonLeafs(root.left) + countNonLeafs(root.right);
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Print all paths to leaves and their details of a binary tree

Given a binary tree print all paths from root node to leaf nodes with their respective lengths and total number of paths in it.
The root node of binary tree is given to you. A path from root to leaf in a tree is a sequence of adjacent nodes from root to any leaf node.
Do not write the whole program, just complete the function **void printAllPaths(Node root)** which takes the address of the root as a parameter and print all details as shown below.
Note: If the tree is empty, do not print anything.

Input Format

First line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.

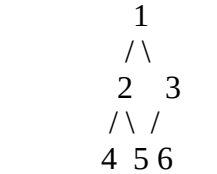
Output

For each test case, print the paths with their length and total paths in new lines.

Constraints

0 <= N <= 10^5
0 <= node data <= 1000

Sample Input



Sample Output

1 2 4 2
1 2 5 2
1 3 6 2
3

Explanation:

First path is from 1 to 4 having 2 edges, so 1 2 4 is path and 2 is length of it.
Similarly for other two leaf nodes. And at last line total number of paths are printed.

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
static int total=0;
```

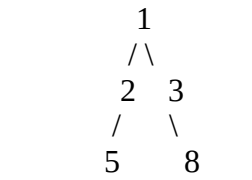

RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
  static void printAllPaths(Node root) {
    // Write your code here
if(root==null) return;
      int arr[]= new int[100];
      solve(root, arr, 0);
      System.out.print(total);
  }
static void solve(Node root, int arr[], int len){
    if(root==null) return;
    arr[len++]=root.data;
    if(root.left==null && root.right==null){
        for(int i=0;i<len;i++){
            System.out.print(arr[i]+" ");
        }
        System.out.println(len-1);
        total++;
        return;
    }
    solve(root.left, arr, len);
    solve(root.right, arr, len);
}
```

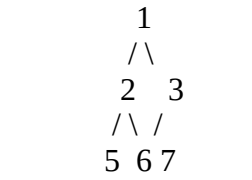

RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find the right node of a given node

Given a binary tree and a key in it, find the node which is on same level and on right of this given key. If no such node present return -1.
Complete the function **findRightSibling()** which takes the address of the root node and an integer key as a parameter and return the right sibling (if exists, otherwise return -1).
þÿInput Format
First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.
Third line contains an integer key whose right sibling is desired.
Output Format
Print the right sibling if any, otherwise print -1.
Sample Input 1
7
1 2 3 5 -1 -1 8
5
Sample Output 1
8
Explanation 1



þÿFor 5, the right sibling is 8.
Sample Input 2
6
1 2 3 5 6 7
7
Sample Output 2
-1
Explanation 2



For 7, there is no node present in its right on the same level.
Therefore, the answer is -1.

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
  static int findRightSibling(Node root, int key) {
    // Write your code here
    if(root==null) return -1;
      Queue <Node> q= new LinkedList<>();
      q.add(root);
      while(!q.isEmpty()){
          int n=q.size();
          for(int i=0;i<n;i++){
              Node curr=q.remove();
              if(curr.data==key){
                  if(i==n-1) return -1;
                  return q.peek().data;
              }
                if(curr.left!=null) q.add(curr.left);
            if(curr.right!=null) q.add(curr.right);
          }
      }
      return -1;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Convert a binary tree into its mirror tree

Given a binary tree, convert it in its mirror image. Mirror of a Binary Tree is another Binary Tree in which left and right children of all non-leaf nodes are interchanged.

Input

The root node of binary tree is given to you. Do not write the whole program, just complete the function findMirror(Node root) which takes the address of the root as a parameter and change the tree in its mirror image.

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

Output

For each test case, print the tree in inorder in new lines.

Sample Input

```
      1
     / \
    2   3
   / \ /
  4 5 6
```

Sample Output

```
     1
    / \
   3   2
   \  / \
    6 5  4
```

Inorder: 3 6 1 5 2 4

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
static Node findMirror(Node root) {
  // Write your code here
  if(root==null) return null;
    Node temp=root.left;
    root.left=root.right;
    root.right=temp;
    findMirror(root.left);
    findMirror(root.right);
```

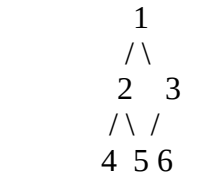

RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    return root;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Print cousins of a given node in Binary Tree

Given a binary tree and a key value from this tree, print all the cousins of this node separated by space. If no cousin exists print -1.
Complete the function **printCousins()** which takes the address of the root node and a key k as a parameter and print the cousins of k separated by space or -1 if no cousin exists.
Input Format
First line contains the total number of nodes, second line contains the node labels separated by space. Third line contains an integer key k whose cousins are desired.
Output Format
For each test case, print the cousins of given key separated by space in new lines.
Sample Input
6
1 2 3 4 5 6
2
Sample Output
-1
Explanation:



The parent of node 2 is 1, who have no siblings, no cousins for node 2.
If key is 6, then 2 is the sibling of parent i.e. 3. So the cousins are 4 and 5þÿ.

Solution:

```
import java.util.*;
class Result {
  static void printCousins(Node root, int k) {
if(root==null || root.data==k){
    System.out.println(-1);
    return;
}
     Queue<Node> q= new LinkedList<>();
     q.add(root);
     while(!q.isEmpty()){
        int n=q.size();
         boolean found=false;
        for(int i=0;i<n;i++){
           Node curr=q.remove();
           if((curr.leftChild!=null && curr.leftChild.data==k) || (curr.rightChild!=null &&
curr.rightChild.data==k)){
              found=true;
        }
           else{
              if(curr.leftChild!=null) q.add(curr.leftChild);
              if(curr.rightChild!=null) q.add(curr.rightChild);
           }
        }
        if(found) break;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
        }
        if(q.isEmpty()){
                System.out.print(-1);
                return;
             }
           while(!q.isEmpty())
                System.out.print(q.remove().data+" ");
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Print nodes in a top view of Binary Tree

Given a binary tree, print top view of the binary tree.
For Example:
[image:https://lh4.googleusercontent.com/6ciAz3U8GM6G1FoGxS5R-6xadLRYegCubYTrdNglg5hV55TT4fzTEes9Mkc6MZMV5M0_93uhVw41jc6QqQo8M6ast5iF7Ms_fJIggUTS447inFfYobB20pjcAE_Inw]

Top view of the given binary tree will be: **4 2 1 3 9**
Complete the function **printTopView()** which takes the address of the root as a parameter and print the tree from top view.
Input Format
First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.
Output Format
For each test case, print the tree nodes from top view separated by space in new lines.
Sample Input
7
1 2 3 4 5 -1 6
Sample Output
4 2 1 3 6
Explanation:

```
      1
     / \
    2   3
   / \   \
  4   5   6
```

So, from top, first node is the left most node i.e. 4, and then 2, 1, 3 and lastly 6.
þÿNote that, 5 is not seen from top.

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
   static TreeMap <Integer, int[]> map= new TreeMap<>();
  static void printTopView(Node root) {
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    // Write your code here
    solve(root, 0, 0);
      for(int x[] :map.values()){
          System.out.print(x[0]+" ");
      }
  }
    static void solve(Node root, int hd, int level){
        if(root==null) return;
        if(!map.containsKey(hd) || level<map.get(hd)[1]){
            map.put(hd, new int[] {root.data, level});
        }
        solve(root.left, hd-1, level+1);
        solve(root.right, hd+1, level+1);
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find out if the tree can be folded or not

Given a binary tree, find out if it can be folded or not. A tree can be folded if left and right sub-trees of root are structure wise mirror images.
Note: A tree with zero or one node is foldable inherently.
Complete the function **isFoldable()** which takes the address of the root as parameter and return 1 if tree can be folded and 0 otherwise.
Input Format
First line contains an integer T, denoting the number of test cases.
For each test case:
First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.
Output Format
For each test case, print 1 if tree is foldable and 0 otherwise, in new lines.
Sample Input 1

```
    1
   / \
  2   3
```

Sample Output 1
1
Sample Input 2

```
     1
    / \
   2   3
  / \
 4  5
```

Sample Output 2
0

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
  static int isFoldable(Node root) {
    // Write your code here
    if(root==null) return 1;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    return Mirror(root.left, root.right)?1:0;
  }
   static boolean Mirror(Node a, Node b){
      if(a==null && b==null) return true;
      if(a==null || b==null) return false;
      return Mirror(a.left , b.right) && Mirror(a.right, b.left);
   }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Given two trees are identical or not

Given two binary trees, find out both are same or not. Two trees are considered same if they have same nodes and same structure.
So complete the function **areSameTree()** which takes the address of the root nodes of two trees as parameters and return **1** if both are same and **0** otherwise.

Input Format

The first line contains the number of testcases, T.
For each Testcase:
First line contains the number of nodes in first tree and second line contains the node labels in level-wise order.
Third line contains the number of nodes in second tree and fourth line contains the node labels in level-wise order.

Output Format

For each test case, print 1 if both tree are same and 0 otherwise.

Sample Input 1

1 // Number of testcases
3
1 2 3
3
1 2 3

Sample Output 1

1

Explanation 1

```
    1          1
   / \        / \
  2   3      2   3
```

As both the trees have same number of nodes and all same labels so trees are same.

Sample Input 2

1 // Number of testcases
5
1 2 3 4 5
3
1 2 3

Sample Output 2

0

Explanation 2

```
    1          1
   / \        / \
  2   3      2   3
 / \
4   5
```

In this case, trees are not same

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
  /*
   * @Input
   *   t1, t2 -> Given the root node of two binary trees
   *
   * @Output
   *   Return 1 if the two binary trees are identical, else return 0
   */
  static int areSameTree(Node t1, Node t2) {
    // Write your code here
    if(t1==null && t2==null) return 1;
      if(t1==null || t2==null) return 0;
      if(t1.data!=t2.data) return 0;
      return (areSameTree(t1.left, t2.left)==1 && areSameTree(t1.right, t2.right)==1)?1 :0;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find maximum depth or height of a binary tree

Height of a tree is defined as the length of the longest downward path from root node to any leaf. If tree is empty, height is considered as -1 and for tree with only one node height is considered as 0.
Your task is that given a binary tree, find out the maximum depth of tree (also called height of tree).
Complete the function **treeHeight()** which takes the address of the root node of tree as parameter and return the height of the tree.

Input Format

First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.

Output Format

For each test case, print the height of tree.

Constraints

0 <= N <= 10^5

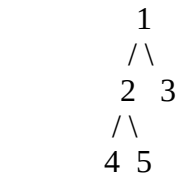
Sample Input

5
1 2 3 4 5

Sample Output

2

Explanation:



The longest path are 1-2-4 and 1-2-5, both have a length of 2, so tree height is 2.

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
  static int treeHeight(Node root) {
    // Write your code here
    if(root==null) return -1;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
        int left=treeHeight(root.left);
        int right=treeHeight(root.right);
        return 1+Math.max(left,right);
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Evaluation of expression tree

Given a expression binary tree with 4 binary operators (+, -, *, /) and integer operands, evaluate it and print the answer.
Complete the function **evaluateTree()** which takes the address of the root node of tree as parameter and return the result of expression.
Note: The nodes with operators are given as ASCII codes of these operators (e.g. 42 for *(multiply), 43 for +(addition), 45 for -(subtraction) & 47 for /(division)).
Input Format
First line contains the total number of nodes and second line contains the node labels separated by space.
Output Format
For each test case, print the result of expression, in new lines.
Sample Input
7
43 42 43 4 5 2 4
Sample Output
26
Explanation:

```
     +
    / \
   *   +
  / \  / \
 4  5 2  4
```

Nodes which are having child nodes are the operator nodes and leaf nodes are the operand nodes to distinguish between.

Solution:

```
/* class Node {
  int data; // data used as key value
  Node leftChild;
  Node rightChild;
  public Node()  {
    data=0;  }
  public Node(int d)  {
    data=d;  }
 } Above class is declared for use. */
class Result {
  static int evaluateTree(Node t1) {
if(t1==null) return 0;
      if(t1.leftChild==null || t1.rightChild==null) return t1.data;
      int a=evaluateTree(t1.leftChild);
      int b=evaluateTree(t1.rightChild);
      switch(t1.data){
         case 42:
            return a*b;
         case 43:
            return a+b;
         case 45:
            return a-b;
         case 47:
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            return a/b;
        }
        return 0;
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Given binary tree is binary search tree or not

A binary tree is called binary search tree if it holds following three properties: -
a. The left subtree of a node contains nodes whose keys are less than that node's key.
b. The right subtree of a node contains nodes whose keys are greater than that node's key.
c. Both the left and right subtrees must also be binary search trees.
Your task is that, given a binary tree check if it is binary search tree or not. Complete the function **isBinarySearchTree()** which takes the address of the root node of tree as parameter and return **1** if the tree is binary search tree and **0** otherwise.

Input Format

First line contains an integer T, denoting the number of test cases.
For each test case:
First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.

Output Format

For each test case, print 0 or 1, in new lines.

Constraints

1 <= T <= 10
0 <= N <= 10^5þÿ

Sample Input

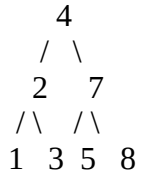
```
1  // testcases
7  // N
4 2 7 1 3 5 8
```

Sample Output

1

Explanation:

Given tree is:



Which satisfies the property of binary search tree, so return value is 1.

Solution:

```
class Result {
  static int isBinarySearchTree(Node root) {
    if(isValid(root, Integer.MIN_VALUE, Integer.MAX_VALUE))
        return 1;
    return 0;
  }
  static boolean isValid(Node root, int min, int max) {
    if(root == null)
        return true;
    if(root.data <= min || root.data >= max)
        return false;
    return isValid(root.left, min, root.data) &&
           isValid(root.right, root.data, max);
  }
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find the kth smallest element in the binary search tree

Given a binary search tree and a number **k**, print the kth smallest number in tree.

Input

The root node of binary search tree is given to you. Do not write the whole program, just complete the function int kSmallest(Node root, int k) which takes the address of the root node of tree and an integer k as parameters and return the kth smallest number from tree.

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

Output

Print the kth smallest number from the binary search tree.

Sample Input

```
      4
     / \
    2   7
   / \  / \
  1  3 5  8

k = 4
k = 6
```

Sample Output

```
4
7
```

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
static int count=0;
static int ans=0;
  static int kSmallest(Node root, int k) {
    // Write your code here
    count=0;
      solve(root,k);
      return ans;
  }
static void solve(Node root, int k){
    if(root==null) return;
    solve(root.left, k);
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    count++;
    if(count==k){
        ans=root.data;
        return;
    }
    solve(root.right, k);
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Convert Level Order Traversal to BST

Given an array of integer elements representing the level order traversal of a binary search tree, create the binary search tree from this array.
Write the function **buildSearchTree()** which takes the array and total number of nodes as parameters and return the root of the binary search tree.

Input Format

First line contains the number of elements in the array(containing the level order traversal) and second line contains the elements separated by space.

þÿOutput Format

Print the tree with inorder traversal.

Sample Input

7
4 2 7 1 3 5 8

Sample Output

1 2 3 4 5 7 8

Explanation:

The tree from above level order traversal will be:

```
      4
     / \
    2   7
   / \ / \
  1  3 5  8
```

Solution:

```
/* class Node {
  int data; // data used as key value
  Node leftChild;
  Node rightChild;
  public Node()  {
    data=0;  }
  public Node(int d)  {
    data=d;  }
 } Above class is declared for use. */
class Result {
  static Node buildSearchTree(int t[], int n) {
    Node root = null;
    // Complete the function body.
for(int i=0;i<n;i++){
    root=insert(root,t[i]);
}
    return(root);
  }
    static Node insert(Node root, int val){
        if(root==null) return new Node(val);
        if(val<root.data) root.leftChild= insert(root.leftChild, val);
         if(val>root.data) root.rightChild= insert(root.rightChild, val);
         return root;
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find a lowest common ancestor of a given two nodes in a binary search tree

The Lowest Common Ancestor (LCA) of two nodes in a Binary Search Tree (BST) is defined as the deepest node from the root which lies in the path of both the nodes from the root.
So, given a binary search tree, find the Lowest Common Ancestor of two given nodes in it.
Complete the function **lowestCommonAncestor()** which takes the address of the root node of tree and two integer keys as parameters and return the lowest common ancestor of these two nodes (For empty tree return -1).
You can assume that both the given keys are present in the tree and are different from each other.

Input Format

First line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.
Third line contains two integers k1 and k2 separated by space.

Output Format

Print the lowest common ancestor of given two nodes from the binary search tree.

Constraints

0 <= N <= 10^5

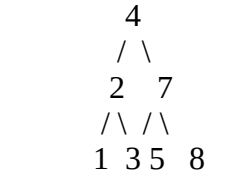
Sample Input

```
7
4 2 7 1 3 5 8
1 5
```

Sample Output

```
4
```

Explanation:



Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
  static int lowestCommonAncestor(Node root, int k1, int k2) {
    // Write your code here
    if(root==null) return -1;
      if(k1>root.data && k2>root.data) return lowestCommonAncestor(root.right, k1, k2);
      if(k1<root.data && k2<root.data) return lowestCommonAncestor(root.left, k1, k2);
      return root.data;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find the floor and ceil of a key in binary search tree

Given a binary search tree, find the floor and ceil of an key in the tree. They are defined as below:
floor value of k: largest node value which is smaller than or equal to k.
ceil value of k: smallest node value which is greater than or equal to k.
Complete the functions **floorOf()** & **ceilOf()** which takes the address of the root node of tree and an integer key as parameters and return the floor and ceil of that key respectively, and -1 if not found.

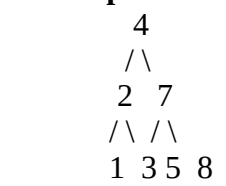
Input Format:
First line contains the number of nodes, second line contains the node labels in level-wise order. Third line contains an integer k whose floor and ceil are desired.

Output Format:
Print the floor and ceil node values of given key from the binary search tree. If not exists, print -1.

Sample Input
7
4 2 7 1 3 5 8
2

Sample Output
2 2

Explanation:



For 2, As 2 is itself present in the tree so 2 will be the floor and ceil of itself.
Furthermore, for 6, the largest node value which is smaller than or equal to 6 is 5, and smallest node value which is greater than or equal to 6 is 7.
And for 9, the largest node value which is smaller than or equal to 9 is 8, and smallest node value which is greater than or equal to 9 is not in the tree so -1 will be printed.

Solution:

```
/* class Node {
  int data; // data used as key value
  Node leftChild;
  Node rightChild;
  public Node()  {
    data=0;  }
  public Node(int d)  {
    data=d;  }
 } Above class is declared for use. */
class Result {
    static int floor=-1;
    static int ceil=-1;
  static int floorOf(Node root, int key) {
if(root==null) return floor;
      if(root.data==key) return root.data;
      if(root.data<key){
          floor=root.data;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
            return floorOf(root.rightChild, key);
        }
        return floorOf(root.leftChild, key);
    }
    static int ceilOf(Node root, int key) {
if(root==null) return ceil;
        if(root.data==key) return root.data;
        if(root.data>key){
            ceil=root.data;
            return ceilOf(root.leftChild, key);
        }
        return ceilOf(root.rightChild, key);
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Find K largest elements in array

Given an array of integer elements and an integer k, print the k largest elements from array separated by space in descending sorted order. For example, if array a = [2, 56, 3, 87, 42, 1, 45, 8, 2, 98] and k=3 then output must be 98 87 56.

Input Format:

First line of input contain T = number of test cases. Each test case contain two lines, in first line, total number of elements in array and value of k are provided and next line will contain the elements.

The array of numbers and integer k is given to you.

Note: Do not write the whole program, just complete the functions void printKLargest(int array[], int k) which takes the array and integer k as parameters and print the k largest elements in descending order separated by space.

Note: Do not read any input from stdin/console. Just complete the functions provided. You can write more functions if required, but just above the given function.

Output Format:

Print the k largest element in descending order separated by space.

Do not print the space after last element.

Sample Input

```
2
10 3
2 56 3 87 42 1 45 8 2 98
6 2
1 87 2 67 3 45
```

Sample Output

```
98 87 56
87 67
```

Solution:

```
static void printKLargest(int array[], int n, int k){
   Arrays.sort(array);  // sort in ascending order
   for(int i = n - 1; i >= n - k; i--){
      System.out.print(array[i]);
      if(i != n - k) System.out.print(" ");
   }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Convert min Heap to max Heap

Given array representation of a min-heap convert it in a max-heap representation. For example, if array a = [13 15 19 16 18 120 110 112 118] the array must be modified as a = [120 118 110 112 18 19 13 15 16].
The array of numbers is given to you. Do not write the whole program, just complete the functions **void modifyMintoMax(int array[], int n)** which takes the array and total number of elements n as parameters and modify the array elements with **heapify()** to convert it in max-heap.
Note: Do not read any input from stdin/console. Just complete the functions provided. You can write more functions if required, but just above the given function.
Input Format:
First line of input contain T = number of test cases.
Each test case contain two lines, in first line, total number of elements in array and next line will contain the elements.
Output Format:
For each test case, print the max-heap elements separated by space in new lines.
Sample Input
2
9
13 15 19 16 18 120 110 112 118
6
1 2 3 4 5 6
Sample Output
120 118 110 112 18 19 13 15 16
6 5 3 4 2 1

Solution:

```
static void modifyMintoMax(int array[], int n)
{
   // Start from last non-leaf node
   for(int i = n/2 - 1; i >= 0; i--){
      maxHeapify(array, n, i);
   }
}
// Max Heapify function
static void maxHeapify(int arr[], int n, int i){
   int largest = i;
   int left = 2*i + 1;
   int right = 2*i + 2;
   if(left < n && arr[left] > arr[largest]){
      largest = left;
   }
   if(right < n && arr[right] > arr[largest]){
      largest = right;
   }
   if(largest != i){
      int temp = arr[i];
      arr[i] = arr[largest];
      arr[largest] = temp;
      maxHeapify(arr, n, largest);
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Check if a given tree is max-heap or not

A max-heap is a kind of tree which must satisfy the property as “Every node’s value should be greater than its children’s value”.
Your task is given level wise array representation of a binary tree, check if it is a max-heap or not.
Complete the functions **isMaxHeap()** which takes the array and total number of elements n as parameters and return 1 if array represents a max-heap and 0 otherwise.

Input Format

First line of input contain T = number of test cases. Each test case contain two lines, in first line, total number of elements in tree and next line will contain the elements in level order fashion.

Output Format

For each test case, print 1 if array represents max-heap or 0 otherwise in new lines.

Sample Input

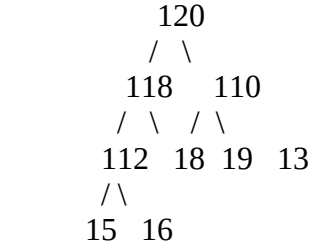
```
2
9
120 118 110 112 18 19 13 15 16
6
1 2 3 4 5 6
```

Sample Output

```
1
0
```

Explanation:

First tree is like below:



It satisfies the properties of max-heap.

Solution:

```
static int isMaxHeap(int array[], int n){
   for(int i = 0; i <= n/2 - 1; i++){
      int left = 2*i + 1;
      int right = 2*i + 2;
      // Check left child
      if(left < n && array[i] < array[left]){
         return 0;
      }
      // Check right child
      if(right < n && array[i] < array[right]){
         return 0;
      }
   }
   return 1;
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Connect the sticks of different lengths with minimum cost

Given n sticks of different length you have to connect them. The cost of connecting sticks is the sum of their length. You have to find the minimum cost for connecting all sticks. For example, if 3 sticks are there having length respectively 3, 6, 1 then we can connect them in many ways like:
Connect 3 and 6 (Cost 9), then we have 2 sticks of length 9 and 1 connect them (cost 10), So total cost is 19.
Connect 3 and 1 (Cost 4), then we have 2 sticks of length 4 and 6 connect them (cost 10), So total cost is 14.
So your function must return 14 in this case.
Complete the function **connectCost()** which takes the array and total number of sticks as input and returns the minimum cost of connecting all these sticks.

Input Format

First line of input contain T = number of test cases.
Each test case contain two lines, in first line, total number of sticks and next line will contain the lengths of each stick.

Output Format

For each test case, print the total cost of connecting all sticks in new lines.

Constraints

1 <= T <= 10
1 <= n <= 10^5
1 <= lengths[i] <= 1000

Sample Input

2
3
3 6 1
4
4 2 3 6

Sample Output

14
29

Solution:

```
class Result
{
  static int connectCost(int lengths[], int n){
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    // Add all sticks
    for(int i = 0; i < n; i++){
        pq.add(lengths[i]);
    }
    int cost = 0;
    while(pq.size() > 1){
        int first = pq.poll();
        int second = pq.poll();
        int sum = first + second;
        cost += sum;
        pq.add(sum);
    }
    return cost;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
}
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Implement Priority Queue using Linked List

Implement the priority queue using linked list representation.

Input Format:

First line of input contains an integer Q denoting the number of queries. A query can be of 1 of the below 2 types: -

(a) 1 item p (a query of this type means insert 'item' into the queue with priority p)

(b) 2 (a query of this type means to delete highest priority element from queue and print it, Assume lowest number has highest priority i.e. 0 is highest priority and 9 is lowest).

Next lines of each test case contains Q queries as shown below.

The functions void enqueue() & int dequeue() will take the head of list and modify list and head appropriately. The head pointer is passed by reference to both of them. You are required to complete the methods given.

Output Format:

The output for each test case will be space separated integers having -1 if the queue is empty else the element deleted from the queue.

Sample Input

```
1   // No. of test cases
4   // No. of queries
1   // Query type insert
10  // value
1   // priority
1   // Query type insert
20  // value
0   // priority
2   // Query type delete
2   // Query type delete
```

Sample Output

```
20 10
```

Explanation:

1st query is 1 10 1, which inserts element 10 with priority 1 in queue,
2nd query is 1 20 0, which inserts element 20 with priority 0 in queue,
3rd query is 2, which will delete the highest priority element and print it i.e. 20
4th query is 2, which will delete the highest priority element and print it i.e. 10

Solution:

```
class PQueueLL
{
  public QueueNode front, rear;
  public void EnQueue(int data, int priority)
  {
    QueueNode newNode = new QueueNode(data, priority);
    // Case 1: Empty queue
    if(front == null){
      front = rear = newNode;
      return;
    }
    // Case 2: Insert at beginning (highest priority)
    if(priority < front.priority){
      newNode.next = front;
      front = newNode;
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
      return;
    }
    // Case 3: Insert in middle or end
    QueueNode temp = front;
    while(temp.next != null && temp.next.priority <= priority){
      temp = temp.next;
    }
    newNode.next = temp.next;
    temp.next = newNode;
    // Update rear if inserted at end
    if(newNode.next == null){
      rear = newNode;
    }
  }
  public int DeQueue()
  {
    // Empty queue
    if(front == null){
      return -1;
    }
    int value = front.data;
    front = front.next;
    // If queue becomes empty
    if(front == null){
      rear = null;
    }
    return value;
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

Aim: Sort an array using heap sort

Given an array of N integers, sort them in ascending order using heap sort. Write the two functions **heapify()** and **heapSort(),** to implement the heap.

Input Format

First line contains the number of elements
Second line contains the elements of array separated by space.

Output Format

Print the elements of sorted array in ascending order.

Constraints

1 <= N <= 10^5
-(10^9) <= arr[i] <= 10^9

Sample Input

7
1 3 5 7 2 4 9

Sample Output

1 2 3 4 5 7 9

Solution:

```
class Result {
  static void heapify(int array[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    if(left < n && array[left] > array[largest]){
      largest = left;
    }
    if(right < n && array[right] > array[largest]){
      largest = right;
    }
    if(largest != i){
      int temp = array[i];
      array[i] = array[largest];
      array[largest] = temp;
      heapify(array, n, largest);
    }
  }
  static void heapSort(int array[], int n) {
    // Step 1: Build Max Heap
    for(int i = n/2 - 1; i >= 0; i--){
      heapify(array, n, i);
    }
    // Step 2: Extract elements
    for(int i = n-1; i > 0; i--){
      // Swap root with last
      int temp = array[0];
      array[0] = array[i];
      array[i] = temp;
      // Heapify reduced heap
      heapify(array, i, 0);
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in

```
    }
  }
}
```


RUPALI MANGLA
rupali0702.becse24@chitkara.edu.in